```
# ===== 文件: xiaozhi-server/app.py =====
import sys
import uuid
import signal
import asyncio
from aioconsole import ainput
from config.settings import load_config
from config.logger import setup_logging
from core.utils.util import get_local_ip, validate_mcp_endpoint
from core.http_server import SimpleHttpServer
from core.websocket_server import WebSocketServer
from core.utils.util import check_ffmpeg_installed
from core.utils.gc_manager import get_gc_manager

TAG = __name__
logger = setup_logging()


async def wait_for_exit() -> None:
    """
    阻塞直到收到 Ctrl-C / SIGTERM。
    - Unix: 使用 add_signal_handler
    - Windows: 依赖 KeyboardInterrupt
    """
    loop = asyncio.get_running_loop()
    stop_event = asyncio.Event()

    if sys.platform != "win32":  # Unix / macOS
        for sig in (signal.SIGINT, signal.SIGTERM):
            loop.add_signal_handler(sig, stop_event.set)
        await stop_event.wait()
    else:
        # Windows：await一个永远pending的fut，
        # 让 KeyboardInterrupt 冒泡到 asyncio.run，以此消除遗留普通线程导致进程退出阻塞的问题
        try:
            await asyncio.Future()
        except KeyboardInterrupt:  # Ctrl-C
            pass


async def monitor_stdin():
    """监控标准输入，消费回车键"""
    while True:
        await ainput()  # 异步等待输入，消费回车


async def main():
    check_ffmpeg_installed()
    config = await load_config()

    # auth_key优先级：配置文件server.auth_key > manager-api.secret > 自动生成
    # auth_key用于jwt认证，比如视觉分析接口的jwt认证、ota接口的token生成与websocket认证
    # 获取配置文件中的auth_key
    auth_key = config["server"].get("auth_key", "")

    # 验证auth_key，无效则尝试使用manager-api.secret
    if not auth_key or len(auth_key) == 0 or "你" in auth_key:
        auth_key = config.get("manager-api", {}).get("secret", "")
        # 验证secret，无效则生成随机密钥
        if not auth_key or len(auth_key) == 0 or "你" in auth_key:
```

```
        auth_key = str(uuid.uuid4().hex)

config["server"]["auth_key"] = auth_key

# 添加 stdin 监控任务
stdin_task = asyncio.create_task(monitor_stdin())

# 启动全局GC管理器（5分钟清理一次）
gc_manager = get_gc_manager(interval_seconds=300)
await gc_manager.start()

# 启动 WebSocket 服务器
ws_server = WebSocketServer(config)
ws_task = asyncio.create_task(ws_server.start())
# 启动 Simple http 服务器
ota_server = SimpleHttpServer(config)
ota_task = asyncio.create_task(ota_server.start())

read_config_from_api = config.get("read_config_from_api", False)
port = int(config["server"].get("http_port", 8003))
if not read_config_from_api:
    logger.bind(tag=TAG).info(
        "OTA接口是\t\thttp://{}:{}/xiaozhi/ota/",
        get_local_ip(),
        port,
    )
logger.bind(tag=TAG).info(
    "视觉分析接口是\thttp://{}:{}/mcp/vision/explain",
    get_local_ip(),
    port,
)
mcp_endpoint = config.get("mcp_endpoint", None)
if mcp_endpoint is not None and "你" not in mcp_endpoint:
    # 校验MCP接入点格式
    if validate_mcp_endpoint(mcp_endpoint):
        logger.bind(tag=TAG).info("mcp接入点是\t{}", mcp_endpoint)
        # 将mcp计入点地址转成调用点
        mcp_endpoint = mcp_endpoint.replace("/mcp/", "/call/")
        config["mcp_endpoint"] = mcp_endpoint
    else:
        logger.bind(tag=TAG).error("mcp接入点不符合规范")
        config["mcp_endpoint"] = "你的接入点 websocket地址"

# 获取WebSocket配置，使用安全的默认值
websocket_port = 8000
server_config = config.get("server", {})
if isinstance(server_config, dict):
    websocket_port = int(server_config.get("port", 8000))

logger.bind(tag=TAG).info(
    "Websocket地址是\tws://{}:{}/xiaozhi/v1/",
    get_local_ip(),
    websocket_port,
)

logger.bind(tag=TAG).info(
    "=======上面的地址是websocket协议地址，请勿用浏览器访问======="
)
logger.bind(tag=TAG).info(
    "如想测试websocket请启动digital-human模块，打开浏览器交互测试"
```

```
    )
    logger.bind(tag=TAG).info(
        "============================================================\n"
    )

    try:
        await wait_for_exit()  # 阻塞直到收到退出信号
    except asyncio.CancelledError:
        print("任务被取消，清理资源中...")
    finally:
        # 停止全局GC管理器
        await gc_manager.stop()

        # 取消所有任务（关键修复点）
        stdin_task.cancel()
        ws_task.cancel()
        if ota_task:
            ota_task.cancel()

        # 等待任务终止（必须加超时）
        await asyncio.wait(
            [stdin_task, ws_task, ota_task] if ota_task else [stdin_task, ws_task],
            timeout=3.0,
            return_when=asyncio.ALL_COMPLETED,
        )
        print("服务器已关闭，程序退出。")


if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("手动中断，程序终止。")
# ===== 文件: xiaozhi-server/config/config_loader.py =====
import os
import asyncio
import yaml
from collections.abc import Mapping
from config.manage_api_client import (
    init_service,
    get_server_config,
    get_agent_models,
    get_correct_words,
    DeviceNotFoundException,
    DeviceBindException,
)


def get_project_dir():
    """获取项目根目录"""
    return os.path.dirname(os.path.dirname(os.path.abspath(__file__))) + "/"


def read_config(config_path):
    with open(config_path, "r", encoding="utf-8") as file:
        config = yaml.safe_load(file)
    return config


async def load_config():
```

```
    """加载配置文件"""
    from core.utils.cache.manager import cache_manager, CacheType

    # 检查缓存
    cached_config = cache_manager.get(CacheType.CONFIG, "main_config")
    if cached_config is not None:
        return cached_config

    default_config_path = get_project_dir() + "config.yaml"
    custom_config_path = get_project_dir() + "data/.config.yaml"

    # 加载默认配置
    default_config = read_config(default_config_path)
    custom_config = read_config(custom_config_path)

    if custom_config.get("manager-api", {}).get("url"):
        config = await get_config_from_api_async(custom_config)
    else:
        # 合并配置
        config = merge_configs(default_config, custom_config)
    # 初始化目录
    ensure_directories(config)

    # 缓存配置
    cache_manager.set(CacheType.CONFIG, "main_config", config)
    return config


async def get_config_from_api_async(config):
    """从Java API获取配置（异步版本）"""
    # 初始化API客户端
    init_service(config)

    # 获取服务器配置
    config_data = await get_server_config()
    if config_data is None:
        raise Exception("Failed to fetch server config from API")

    config_data["read_config_from_api"] = True
    config_data["manager-api"] = {
        "url": config["manager-api"].get("url", ""),
        "secret": config["manager-api"].get("secret", ""),
    }
    auth_enabled = config_data.get("server", {}).get("auth", {}).get("enabled", False)
    # server的配置以本地为准
    if config.get("server"):
        config_data["server"] = {
            "ip": config["server"].get("ip", ""),
            "port": config["server"].get("port", ""),
            "http_port": config["server"].get("http_port", ""),
            "vision_explain": config["server"].get("vision_explain", ""),
            "auth_key": config["server"].get("auth_key", ""),
        }
    config_data["server"]["auth"] = {"enabled": auth_enabled}
    # 如果服务器没有prompt_template，则从本地配置读取
    if not config_data.get("prompt_template"):
        config_data["prompt_template"] = config.get("prompt_template")
    return config_data
```

```
async def get_private_config_from_api(config, device_id, client_id):
    """从Java API获取私有配置"""
    results = await asyncio.gather(
        get_agent_models(device_id, client_id, config["selected_module"]),
        get_correct_words(device_id),
        return_exceptions=True,
    )
    agent_result = results[0]
    correct_words = results[1] if not isinstance(results[1], Exception) else None

    # 抛出业务异常
    if isinstance(agent_result, DeviceNotFoundException):
        raise agent_result
    if isinstance(agent_result, DeviceBindException):
        raise agent_result

    private_config = agent_result if not isinstance(agent_result, Exception) else {}
    if correct_words:
        private_config["correct_words"] = correct_words
    return private_config


def ensure_directories(config):
    """确保所有配置路径存在"""
    dirs_to_create = set()
    project_dir = get_project_dir() # 获取项目根目录
    # 日志文件目录
    log_dir = config.get("log", {}).get("log_dir", "tmp")
    dirs_to_create.add(os.path.join(project_dir, log_dir))

    # ASR/TTS模块输出目录
    for module in ["ASR", "TTS"]:
        if config.get(module) is None:
            continue
        for provider in config.get(module, {}).values():
            output_dir = provider.get("output_dir", "")
            if output_dir:
                dirs_to_create.add(output_dir)

    # 根据selected_module创建模型目录
    selected_modules = config.get("selected_module", {})
    for module_type in ["ASR", "LLM", "TTS"]:
        selected_provider = selected_modules.get(module_type)
        if not selected_provider:
            continue
        if config.get(module_type) is None:
            continue
        if config.get(selected_provider) is None:
            continue
        provider_config = config.get(module_type, {}).get(selected_provider, {})
        output_dir = provider_config.get("output_dir")
        if output_dir:
            full_model_dir = os.path.join(project_dir, output_dir)
            dirs_to_create.add(full_model_dir)

    # 统一创建目录（保留原data目录创建）
    for dir_path in dirs_to_create:
        try:
            os.makedirs(dir_path, exist_ok=True)
        except PermissionError:
```

```
        print(f"警告：无法创建目录 {dir_path}，请检查写入权限")


def merge_configs(default_config, custom_config):
    """
    递归合并配置，custom_config优先级更高

    Args:
        default_config: 默认配置
        custom_config: 用户自定义配置

    Returns:
        合并后的配置
    """
    if not isinstance(default_config, Mapping) or not isinstance(
        custom_config, Mapping
    ):
        return custom_config

    merged = dict(default_config)

    for key, value in custom_config.items():
        if (
            key in merged
            and isinstance(merged[key], Mapping)
            and isinstance(value, Mapping)
        ):
            merged[key] = merge_configs(merged[key], value)
        else:
            merged[key] = value

    return merged
# ===== 文件: xiaozhi-server/config/logger.py =====
import os
import sys
import asyncio
from loguru import logger
from config.config_loader import load_config
from config.settings import check_config_file
from core.utils.cache.manager import cache_manager, CacheType

SERVER_VERSION = "0.9.6"
_logger_initialized = False


def get_module_abbreviation(module_name, module_dict):
    """获取模块名称的缩写，如果为空则返回00
    如果名称中包含下划线，则返回下划线后面的前两个字符
    """
    module_value = module_dict.get(module_name, "")
    if not module_value:
        return "00"
    if "_" in module_value:
        parts = module_value.split("_")
        return parts[-1][:2] if parts[-1] else "00"
    return module_value[:2]


def build_module_string(selected_module):
    """构建模块字符串"""
```

```
    return (
        get_module_abbreviation("VAD", selected_module)
        + get_module_abbreviation("ASR", selected_module)
        + get_module_abbreviation("LLM", selected_module)
        + get_module_abbreviation("TTS", selected_module)
        + get_module_abbreviation("Memory", selected_module)
        + get_module_abbreviation("Intent", selected_module)
        + get_module_abbreviation("VLLM", selected_module)
    )


def formatter(record):
    """为没有 tag 的日志添加默认值，并处理动态模块字符串"""
    record["extra"].setdefault("tag", record["name"])
    # 如果没有设置 selected_module，使用默认值
    record["extra"].setdefault("selected_module", "00000000000000")
    # 将 selected_module 从 extra 提取到顶级，以支持 {selected_module} 格式
    record["selected_module"] = record["extra"]["selected_module"]
    return record["message"]


def setup_logging(config=None):
    """从配置文件中读取日志配置，并设置日志输出格式和级别"""
    if config is None:
        check_config_file()
        # 先查缓存，避免在 async 上下文中重复 await load_config
        config = cache_manager.get(CacheType.CONFIG, "main_config")
        if config is None:
            # 缓存也没有（理论上不该发生），才走 asyncio.run
            config = asyncio.run(load_config())
    log_config = config["log"]
    global _logger_initialized

    # 第一次初始化时配置日志
    if not _logger_initialized:
        # 使用默认的模块字符串进行初始化
        logger.configure(
            extra={
                "selected_module": log_config.get("selected_module", "00000000000000"),
            }
        )

        log_format = log_config.get(
            "log_format",
            "<green>{time:YYMMDD HH:mm:ss}</green>[{version}_{extra[selected_module]}][<light-blue>{extr
    a[tag]}</light-blue>]-<level>{level}</level>-<cyan>{file}:{line}</cyan>-<light-green>{message}</ligh
    t-green>",
        )
        log_format_file = log_config.get(
            "log_format_file",
            "{time:YYYY-MM-DD HH:mm:ss} - {version}_{extra[selected_module]} - {name} - {level} - {extra
    [tag]} - {file}:{line} - {message}",
        )
        log_format = log_format.replace("{version}", SERVER_VERSION)
        log_format_file = log_format_file.replace("{version}", SERVER_VERSION)

        log_level = log_config.get("log_level", "INFO")
        log_dir = log_config.get("log_dir", "tmp")
        log_file = log_config.get("log_file", "server.log")
        data_dir = log_config.get("data_dir", "data")
```

```
        log_to_file = log_config.get("log_to_file", True)  # 生产环境容器可设为 false，仅输出到 stdout

        if log_to_file:
            os.makedirs(log_dir, exist_ok=True)
        os.makedirs(data_dir, exist_ok=True)

        # 配置日志输出
        logger.remove()

        # 输出到控制台
        logger.add(sys.stdout, format=log_format, level=log_level, filter=formatter)

        # 输出到文件 - 容器环境可关闭（log_to_file: false），由运行时采集 stdout
        if log_to_file:
            log_file_path = os.path.join(log_dir, log_file)
            logger.add(
                log_file_path,
                format=log_format_file,
                level=log_level,
                filter=formatter,
                rotation="10 MB",  # 每个文件最大10MB
                retention="30 days",  # 保留30天
                compression=None,
                encoding="utf-8",
                enqueue=True,  # 异步安全
                backtrace=True,
                diagnose=True,
            )
        _logger_initialized = True  # 标记为已初始化

    return logger


def create_connection_logger(selected_module_str):
    """为连接创建独立的日志器，绑定特定的模块字符串"""
    return logger.bind(selected_module=selected_module_str)
# ===== 文件: xiaozhi-server/config/manage_api_client.py =====
import os
import base64
from typing import Optional, Dict

import httpx

TAG = __name__


class DeviceNotFoundException(Exception):
    pass


class DeviceBindException(Exception):
    def __init__(self, bind_code):
        self.bind_code = bind_code
        super().__init__(f"设备绑定异常，绑定码: {bind_code}")


class ManageApiClient:
    _instance = None
    _async_clients = {}  # 为每个事件循环存储独立的客户端
    _secret = None
```

```
def __new__(cls, config):
    """单例模式确保全局唯一实例，并支持传入配置参数"""
    if cls._instance is None:
        cls._instance = super().__new__(cls)
        cls._init_client(config)
    return cls._instance

@classmethod
def _init_client(cls, config):
    """初始化配置（延迟创建客户端）"""
    cls.config = config.get("manager-api")

    if not cls.config:
        raise Exception("manager-api配置错误")

    if not cls.config.get("url") or not cls.config.get("secret"):
        raise Exception("manager-api的url或secret配置错误")

    if "你" in cls.config.get("secret"):
        raise Exception("请先配置manager-api的secret")

    cls._secret = cls.config.get("secret")
    cls.free_daily_chat_limit = config.get("chat_quota", {}).get("free_daily_limit", 30)
    cls.max_retries = cls.config.get("max_retries", 6)  # 最大重试次数
    cls.retry_delay = cls.config.get("retry_delay", 10)  # 初始重试延迟(秒)
    # 不在这里创建 AsyncClient，延迟到实际使用时创建
    cls._async_clients = {}

@classmethod
async def _ensure_async_client(cls):
    """确保异步客户端已创建（为每个事件循环创建独立的客户端）"""
    import asyncio

    try:
        loop = asyncio.get_running_loop()
        loop_id = id(loop)

        # 为每个事件循环创建独立的客户端
        if loop_id not in cls._async_clients:
            # 服务端可能主动关闭连接，httpx 连接池无法正确检测和清理
            limits = httpx.Limits(
                max_keepalive_connections=0,  # 禁用 keep-alive，每次都新建连接
            )
            cls._async_clients[loop_id] = httpx.AsyncClient(
                base_url=cls.config.get("url"),
                headers={
                    "User-Agent": f"PythonClient/2.0 (PID:{os.getpid()})",
                    "Accept": "application/json",
                    "Authorization": "Bearer " + cls._secret,
                },
                timeout=cls.config.get("timeout", 30),
                limits=limits,  # 使用限制
                trust_env=False,
            )
        return cls._async_clients[loop_id]
    except RuntimeError:
        # 如果没有运行中的事件循环，创建一个临时的
        raise Exception("必须在异步上下文中调用")
```

```
@classmethod
async def _async_request(cls, method: str, endpoint: str, **kwargs) -> Dict:
    """发送单次异步HTTP请求并处理响应"""
    # 确保客户端已创建
    client = await cls._ensure_async_client()
    endpoint = endpoint.lstrip("/")
    response = None
    try:
        response = await client.request(method, endpoint, **kwargs)
        response.raise_for_status()

        result = response.json()

        # 处理API返回的业务错误
        if result.get("code") == 10041:
            raise DeviceNotFoundException(result.get("msg"))
        elif result.get("code") == 10042:
            raise DeviceBindException(result.get("msg"))
        elif result.get("code") != 0:
            raise Exception(f"API返回错误: {result.get('msg', '未知错误')}")

        # 返回成功数据
        return result.get("data") if result.get("code") == 0 else None
    finally:
        # 确保响应被关闭（即使异常也会执行）
        if response is not None:
            await response.aclose()

@classmethod
def _should_retry(cls, exception: Exception) -> bool:
    """判断异常是否应该重试"""
    # 网络连接相关错误
    if isinstance(
        exception, (httpx.ConnectError, httpx.TimeoutException, httpx.NetworkError)
    ):
        return True

    # HTTP状态码错误
    if isinstance(exception, httpx.HTTPStatusError):
        status_code = exception.response.status_code
        return status_code in [408, 429, 500, 502, 503, 504]

    return False

@classmethod
async def _execute_async_request(cls, method: str, endpoint: str, **kwargs) -> Dict:
    """带重试机制的异步请求执行器"""
    import asyncio

    retry_count = 0

    while retry_count <= cls.max_retries:
        try:
            # 执行异步请求
            return await cls._async_request(method, endpoint, **kwargs)
        except Exception as e:
            # 判断是否应该重试
            if retry_count < cls.max_retries and cls._should_retry(e):
                retry_count += 1
                print(
```

```
                f"{method} {endpoint} 异步请求失败，将在 {cls.retry_delay:.1f} 秒后进行第 {retry
_count} 次重试"
                )
                await asyncio.sleep(cls.retry_delay)
                continue
            else:
                # 不重试，直接抛出异常
                raise

    @classmethod
    def safe_close(cls):
        """安全关闭所有异步连接池"""
        import asyncio

        for client in list(cls._async_clients.values()):
            try:
                asyncio.run(client.aclose())
            except Exception:
                pass
        cls._async_clients.clear()
        cls._instance = None


async def get_server_config() -> Optional[Dict]:
    """获取服务器基础配置"""
    return await ManageApiClient._instance._execute_async_request(
        "POST", "/config/server-base"
    )


async def get_agent_models(
    mac_address: str, client_id: str, selected_module: Dict
) -> Optional[Dict]:
    """获取代理模型配置"""
    return await ManageApiClient._instance._execute_async_request(
        "POST",
        "/config/agent-models",
        json={
            "macAddress": mac_address,
            "clientId": client_id,
            "selectedModule": selected_module,
        },
    )


async def get_correct_words(mac_address: str) -> Optional[Dict]:
    """获取智能体替换词"""
    try:
        return await ManageApiClient._instance._execute_async_request(
            "POST", "/config/correct-words",
            json={"macAddress": mac_address}
        )
    except Exception as e:
        print(f"获取替换词失败: {e}")
        return None


async def generate_and_save_chat_summary(session_id: str) -> Optional[Dict]:
    """生成并保存聊天记录总结"""
    try:
```

```
        return await ManageApiClient._instance._execute_async_request(
            "POST",
            f"/agent/chat-summary/{session_id}/save",
        )
    except Exception as e:
        print(f"生成并保存聊天记录总结失败: {e}")
        return None


async def generate_and_save_chat_title(session_id: str) -> Optional[Dict]:
    """生成并保存聊天标题"""
    try:
        return await ManageApiClient._instance._execute_async_request(
            "POST",
            f"/agent/chat-title/{session_id}/generate",
        )
    except Exception as e:
        print(f"生成并保存聊天标题失败: {e}")
        return None


async def report(
    mac_address: str, session_id: str, chat_type: int, content: str, audio, report_time
) -> Optional[Dict]:
    """异步聊天记录上报"""
    if not content or not ManageApiClient._instance:
        return None
    try:
        return await ManageApiClient._instance._execute_async_request(
            "POST",
            f"/agent/chat-history/report",
            json={
                "macAddress": mac_address,
                "sessionId": session_id,
                "chatType": chat_type,
                "content": content,
                "reportTime": report_time,
                "audioBase64": (
                    base64.b64encode(audio).decode("utf-8") if audio else None
                ),
            },
        )
    except Exception as e:
        print(f"TTS上报失败: {e}")
        return None


async def lookup_address_book(caller_mac: str, nickname: str) -> Optional[Dict]:
    """根据昵称查找目标设备"""
    if not ManageApiClient._instance:
        return None
    try:
        return await ManageApiClient._instance._execute_async_request(
            "GET",
            f"/device/address-book/lookup?callerMac={caller_mac}&nickname={nickname}",
        )
    except Exception as e:
        print(f"通讯录查找失败: {e}")
        return None
```

```
async def check_chat_quota(mac_address: str) -> Optional[Dict]:
    """检查聊天配额（先加后查，原子操作）"""
    if not ManageApiClient._instance:
        return None
    try:
        return await ManageApiClient._instance._execute_async_request(
            "POST",
            "/config/chat-quota/check",
            json={"macAddress": mac_address},
        )
    except Exception as e:
        print(f"聊天配额检查失败: {e}")
        # 失败时保守放行，不影响用户体验
        return {"allowed": True, "remaining": -1, "total": ManageApiClient.free_daily_chat_limit}


def init_service(config):
    ManageApiClient(config)


def manage_api_http_safe_close():
    ManageApiClient.safe_close()
# ===== 文件: xiaozhi-server/config/settings.py =====
import os
import asyncio
from config.config_loader import read_config, get_project_dir, load_config


default_config_file = "config.yaml"
config_file_valid = False


def check_config_file():
    global config_file_valid
    if config_file_valid:
        return
    """
    简化的配置检查，仅提示用户配置文件的使用情况
    """
    custom_config_file = get_project_dir() + "data/." + default_config_file
    if not os.path.exists(custom_config_file):
        raise FileNotFoundError(
            "找不到data/.config.yaml文件，请按教程确认该配置文件是否存在"
        )

    # 检查是否从API读取配置
    config = asyncio.run(load_config())
    if config.get("read_config_from_api", False):
        print("从API读取配置")
        old_config_origin = read_config(custom_config_file)
        if old_config_origin.get("selected_module") is not None:
            error_msg = "您的配置文件好像既包含智控台的配置又包含本地配置：\n"
            error_msg += "\n建议您：\n"
            error_msg += "1、将根目录的config_from_api.yaml文件复制到data下，重命名为.config.yaml\n"
            error_msg += "2、按教程配置好接口地址和密钥\n"
            raise ValueError(error_msg)
    config_file_valid = True
# ===== 文件: xiaozhi-server/models/snakers4_silero-vad/hubconf.py =====
dependencies = ['torch', 'torchaudio']
```

```
import torch
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), 'src'))
from silero_vad.utils_vad import (init_jit_model,
                                  get_speech_timestamps,
                                  save_audio,
                                  read_audio,
                                  VADIterator,
                                  collect_chunks,
                                  OnnxWrapper)


def versiontuple(v):
    splitted = v.split('+')[0].split(".")
    version_list = []
    for i in splitted:
        try:
            version_list.append(int(i))
        except:
            version_list.append(0)
    return tuple(version_list)


def silero_vad(onnx=False, force_onnx_cpu=False, opset_version=16):
    """Silero Voice Activity Detector
    Returns a model with a set of utils
    Please see https://github.com/snakers4/silero-vad for usage examples
    """
    available_ops = [15, 16]
    if onnx and opset_version not in available_ops:
        raise Exception(f'Available ONNX opset_version: {available_ops}')

    if not onnx:
        installed_version = torch.__version__
        supported_version = '1.12.0'
        if versiontuple(installed_version) < versiontuple(supported_version):
            raise Exception(f'Please install torch {supported_version} or greater ({installed_version} i
    nstalled)')

    model_dir = os.path.join(os.path.dirname(__file__), 'src', 'silero_vad', 'data')
    if onnx:
        if opset_version == 16:
            model_name = 'silero_vad.onnx'
        else:
            model_name = f'silero_vad_16k_op{opset_version}.onnx'
        model = OnnxWrapper(os.path.join(model_dir, model_name), force_onnx_cpu)
    else:
        model = init_jit_model(os.path.join(model_dir, 'silero_vad.jit'))
    utils = (get_speech_timestamps,
             save_audio,
             read_audio,
             VADIterator,
             collect_chunks)

    return model, utils
# ===== 文件: xiaozhi-server/models/snakers4_silero-vad/src/silero_vad/__init__.py =====
from importlib.metadata import version
try:
    __version__ = version(__name__)
```

```
except:
    pass

from silero_vad.model import load_silero_vad
from silero_vad.utils_vad import (get_speech_timestamps,
                                  save_audio,
                                  read_audio,
                                  VADIterator,
                                  collect_chunks)
# ===== 文件: xiaozhi-server/models/snakers4_silero-vad/src/silero_vad/model.py =====
from .utils_vad import init_jit_model, OnnxWrapper
import torch
torch.set_num_threads(1)


def load_silero_vad(onnx=False, opset_version=16):
    available_ops = [15, 16]
    if onnx and opset_version not in available_ops:
        raise Exception(f'Available ONNX opset_version: {available_ops}')

    if onnx:
        if opset_version == 16:
            model_name = 'silero_vad.onnx'
        else:
            model_name = f'silero_vad_16k_op{opset_version}.onnx'
    else:
        model_name = 'silero_vad.jit'
    package_path = "silero_vad.data"

    try:
        import importlib_resources as impresources
        model_file_path = str(impresources.files(package_path).joinpath(model_name))
    except:
        from importlib import resources as impresources
        try:
            with impresources.path(package_path, model_name) as f:
                model_file_path = f
        except:
            model_file_path = str(impresources.files(package_path).joinpath(model_name))

    if onnx:
        model = OnnxWrapper(model_file_path, force_onnx_cpu=True)
    else:
        model = init_jit_model(model_file_path)

    return model
# ===== 文件: xiaozhi-server/models/snakers4_silero-vad/src/silero_vad/utils_vad.py =====
import torch
import torchaudio
from typing import Callable, List
import warnings

languages = ['ru', 'en', 'de', 'es']


class OnnxWrapper():

    def __init__(self, path, force_onnx_cpu=False):
        import numpy as np
        global np
```

```
    import onnxruntime

    opts = onnxruntime.SessionOptions()
    opts.inter_op_num_threads = 1
    opts.intra_op_num_threads = 1

    if force_onnx_cpu and 'CPUExecutionProvider' in onnxruntime.get_available_providers():
        self.session = onnxruntime.InferenceSession(path, providers=['CPUExecutionProvider'], sess_o
ptions=opts)
    else:
        self.session = onnxruntime.InferenceSession(path, sess_options=opts)

    self.reset_states()
    if '16k' in path:
        warnings.warn('This model support only 16000 sampling rate!')
        self.sample_rates = [16000]
    else:
        self.sample_rates = [8000, 16000]

def _validate_input(self, x, sr: int):
    if x.dim() == 1:
        x = x.unsqueeze(0)
    if x.dim() > 2:
        raise ValueError(f"Too many dimensions for input audio chunk {x.dim()}")

    if sr != 16000 and (sr % 16000 == 0):
        step = sr // 16000
        x = x[:,::step]
        sr = 16000

    if sr not in self.sample_rates:
        raise ValueError(f"Supported sampling rates: {self.sample_rates} (or multiply of 16000)")
    if sr / x.shape[1] > 31.25:
        raise ValueError("Input audio chunk is too short")

    return x, sr

def reset_states(self, batch_size=1):
    self._state = torch.zeros((2, batch_size, 128)).float()
    self._context = torch.zeros(0)
    self._last_sr = 0
    self._last_batch_size = 0

def __call__(self, x, sr: int):

    x, sr = self._validate_input(x, sr)
    num_samples = 512 if sr == 16000 else 256

    if x.shape[-1] != num_samples:
        raise ValueError(f"Provided number of samples is {x.shape[-1]} (Supported values: 256 for 80
00 sample rate, 512 for 16000)")

    batch_size = x.shape[0]
    context_size = 64 if sr == 16000 else 32

    if not self._last_batch_size:
        self.reset_states(batch_size)
    if (self._last_sr) and (self._last_sr != sr):
        self.reset_states(batch_size)
    if (self._last_batch_size) and (self._last_batch_size != batch_size):
```

```
            self.reset_states(batch_size)

        if not len(self._context):
            self._context = torch.zeros(batch_size, context_size)

        x = torch.cat([self._context, x], dim=1)
        if sr in [8000, 16000]:
            ort_inputs = {'input': x.numpy(), 'state': self._state.numpy(), 'sr': np.array(sr, dtype='in
    t64')}
            ort_outs = self.session.run(None, ort_inputs)
            out, state = ort_outs
            self._state = torch.from_numpy(state)
        else:
            raise ValueError()

        self._context = x[..., -context_size:]
        self._last_sr = sr
        self._last_batch_size = batch_size

        out = torch.from_numpy(out)
        return out

    def audio_forward(self, x, sr: int):
        outs = []
        x, sr = self._validate_input(x, sr)
        self.reset_states()
        num_samples = 512 if sr == 16000 else 256

        if x.shape[1] % num_samples:
            pad_num = num_samples - (x.shape[1] % num_samples)
            x = torch.nn.functional.pad(x, (0, pad_num), 'constant', value=0.0)

        for i in range(0, x.shape[1], num_samples):
            wavs_batch = x[:, i:i+num_samples]
            out_chunk = self.__call__(wavs_batch, sr)
            outs.append(out_chunk)

        stacked = torch.cat(outs, dim=1)
        return stacked.cpu()


class Validator():
    def __init__(self, url, force_onnx_cpu):
        self.onnx = True if url.endswith('.onnx') else False
        torch.hub.download_url_to_file(url, 'inf.model')
        if self.onnx:
            import onnxruntime
            if force_onnx_cpu and 'CPUExecutionProvider' in onnxruntime.get_available_providers():
                self.model = onnxruntime.InferenceSession('inf.model', providers=['CPUExecutionProvider'
    ])
            else:
                self.model = onnxruntime.InferenceSession('inf.model')
        else:
            self.model = init_jit_model(model_path='inf.model')

    def __call__(self, inputs: torch.Tensor):
        with torch.no_grad():
            if self.onnx:
                ort_inputs = {'input': inputs.cpu().numpy()}
                outs = self.model.run(None, ort_inputs)
```

```
            outs = [torch.Tensor(x) for x in outs]
        else:
            outs = self.model(inputs)

    return outs


def read_audio(path: str,
               sampling_rate: int = 16000):
    list_backends = torchaudio.list_audio_backends()

    assert len(list_backends) > 0, 'The list of available backends is empty, please install backend manu
    ally. \
                                   \n Recommendations: \n \tSox (UNIX OS) \n \tSoundfile (Windows OS, U
    NIX OS) \n \tffmpeg (Windows OS, UNIX OS)'

    try:
        effects = [
            ['channels', '1'],
            ['rate', str(sampling_rate)]
        ]

        wav, sr = torchaudio.sox_effects.apply_effects_file(path, effects=effects)
    except:
        wav, sr = torchaudio.load(path)

        if wav.size(0) > 1:
            wav = wav.mean(dim=0, keepdim=True)

        if sr != sampling_rate:
            transform = torchaudio.transforms.Resample(orig_freq=sr,
                                                       new_freq=sampling_rate)
            wav = transform(wav)
            sr = sampling_rate

    assert sr == sampling_rate
    return wav.squeeze(0)


def save_audio(path: str,
               tensor: torch.Tensor,
               sampling_rate: int = 16000):
    torchaudio.save(path, tensor.unsqueeze(0), sampling_rate, bits_per_sample=16)


def init_jit_model(model_path: str,
                   device=torch.device('cpu')):
    model = torch.jit.load(model_path, map_location=device)
    model.eval()
    return model


def make_visualization(probs, step):
    import pandas as pd
    pd.DataFrame({'probs': probs},
                 index=[x * step for x in range(len(probs))]).plot(figsize=(16, 8),
                 kind='area', ylim=[0, 1.05], xlim=[0, len(probs) * step],
                 xlabel='seconds',
                 ylabel='speech probability',
                 colormap='tab20')
```

```
@torch.no_grad()
def get_speech_timestamps(audio: torch.Tensor,
                          model,
                          threshold: float = 0.5,
                          sampling_rate: int = 16000,
                          min_speech_duration_ms: int = 250,
                          max_speech_duration_s: float = float('inf'),
                          min_silence_duration_ms: int = 100,
                          speech_pad_ms: int = 30,
                          return_seconds: bool = False,
                          visualize_probs: bool = False,
                          progress_tracking_callback: Callable[[float], None] = None,
                          neg_threshold: float = None,
                          window_size_samples: int = 512,):

    """
    This method is used for splitting long audios into speech chunks using silero VAD

    Parameters
    ----------
    audio: torch.Tensor, one dimensional
        One dimensional float torch.Tensor, other types are casted to torch if possible

    model: preloaded .jit/.onnx silero VAD model

    threshold: float (default - 0.5)
        Speech threshold. Silero VAD outputs speech probabilities for each audio chunk, probabilities AB
    OVE this value are considered as SPEECH.
        It is better to tune this parameter for each dataset separately, but "lazy" 0.5 is pretty good f
    or most datasets.

    sampling_rate: int (default - 16000)
        Currently silero VAD models support 8000 and 16000 (or multiply of 16000) sample rates

    min_speech_duration_ms: int (default - 250 milliseconds)
        Final speech chunks shorter min_speech_duration_ms are thrown out

    max_speech_duration_s: int (default -  inf)
        Maximum duration of speech chunks in seconds
        Chunks longer than max_speech_duration_s will be split at the timestamp of the last silence that
     lasts more than 100ms (if any), to prevent agressive cutting.
        Otherwise, they will be split aggressively just before max_speech_duration_s.

    min_silence_duration_ms: int (default - 100 milliseconds)
        In the end of each speech chunk wait for min_silence_duration_ms before separating it

    speech_pad_ms: int (default - 30 milliseconds)
        Final speech chunks are padded by speech_pad_ms each side

    return_seconds: bool (default - False)
        whether return timestamps in seconds (default - samples)

    visualize_probs: bool (default - False)
        whether draw prob hist or not

    progress_tracking_callback: Callable[[float], None] (default - None)
        callback function taking progress in percents as an argument
```

```
neg_threshold: float (default = threshold - 0.15)
    Negative threshold (noise or exit threshold). If model's current state is SPEECH, values BELOW t
his value are considered as NON-SPEECH.

window_size_samples: int (default - 512 samples)
    !!! DEPRECATED, DOES NOTHING !!!

Returns
----------
speeches: list of dicts
    list containing ends and beginnings of speech chunks (samples or seconds based on return_seconds
)
"""

if not torch.is_tensor(audio):
    try:
        audio = torch.Tensor(audio)
    except:
        raise TypeError("Audio cannot be casted to tensor. Cast it manually")

if len(audio.shape) > 1:
    for i in range(len(audio.shape)):  # trying to squeeze empty dimensions
        audio = audio.squeeze(0)
    if len(audio.shape) > 1:
        raise ValueError("More than one dimension in audio. Are you trying to process audio with 2 c
hannels?")

if sampling_rate > 16000 and (sampling_rate % 16000 == 0):
    step = sampling_rate // 16000
    sampling_rate = 16000
    audio = audio[::step]
    warnings.warn('Sampling rate is a multiply of 16000, casting to 16000 manually!')
else:
    step = 1

if sampling_rate not in [8000, 16000]:
    raise ValueError("Currently silero VAD models support 8000 and 16000 (or multiply of 16000) samp
le rates")

window_size_samples = 512 if sampling_rate == 16000 else 256

model.reset_states()
min_speech_samples = sampling_rate * min_speech_duration_ms / 1000
speech_pad_samples = sampling_rate * speech_pad_ms / 1000
max_speech_samples = sampling_rate * max_speech_duration_s - window_size_samples - 2 * speech_pad_sa
mples
min_silence_samples = sampling_rate * min_silence_duration_ms / 1000
min_silence_samples_at_max_speech = sampling_rate * 98 / 1000

audio_length_samples = len(audio)

speech_probs = []
for current_start_sample in range(0, audio_length_samples, window_size_samples):
    chunk = audio[current_start_sample: current_start_sample + window_size_samples]
    if len(chunk) < window_size_samples:
        chunk = torch.nn.functional.pad(chunk, (0, int(window_size_samples - len(chunk))))
    speech_prob = model(chunk, sampling_rate).item()
    speech_probs.append(speech_prob)
    # caculate progress and seng it to callback function
    progress = current_start_sample + window_size_samples
```

```
    if progress > audio_length_samples:
        progress = audio_length_samples
    progress_percent = (progress / audio_length_samples) * 100
    if progress_tracking_callback:
        progress_tracking_callback(progress_percent)

triggered = False
speeches = []
current_speech = {}

if neg_threshold is None:
    neg_threshold = max(threshold - 0.15, 0.01)
temp_end = 0  # to save potential segment end (and tolerate some silence)
prev_end = next_start = 0  # to save potential segment limits in case of maximum segment size reache
d

for i, speech_prob in enumerate(speech_probs):
    if (speech_prob >= threshold) and temp_end:
        temp_end = 0
        if next_start < prev_end:
            next_start = window_size_samples * i

    if (speech_prob >= threshold) and not triggered:
        triggered = True
        current_speech['start'] = window_size_samples * i
        continue

    if triggered and (window_size_samples * i) - current_speech['start'] > max_speech_samples:
        if prev_end:
            current_speech['end'] = prev_end
            speeches.append(current_speech)
            current_speech = {}
            if next_start < prev_end:  # previously reached silence (< neg_thres) and is still not s
peech (< thres)
                triggered = False
            else:
                current_speech['start'] = next_start
            prev_end = next_start = temp_end = 0
        else:
            current_speech['end'] = window_size_samples * i
            speeches.append(current_speech)
            current_speech = {}
            prev_end = next_start = temp_end = 0
            triggered = False
            continue

    if (speech_prob < neg_threshold) and triggered:
        if not temp_end:
            temp_end = window_size_samples * i
        if ((window_size_samples * i) - temp_end) > min_silence_samples_at_max_speech:  # condition
to avoid cutting in very short silence
            prev_end = temp_end
        if (window_size_samples * i) - temp_end < min_silence_samples:
            continue
        else:
            current_speech['end'] = temp_end
            if (current_speech['end'] - current_speech['start']) > min_speech_samples:
                speeches.append(current_speech)
            current_speech = {}
            prev_end = next_start = temp_end = 0
```

```
            triggered = False
            continue

    if current_speech and (audio_length_samples - current_speech['start']) > min_speech_samples:
        current_speech['end'] = audio_length_samples
        speeches.append(current_speech)

    for i, speech in enumerate(speeches):
        if i == 0:
            speech['start'] = int(max(0, speech['start'] - speech_pad_samples))
        if i != len(speeches) - 1:
            silence_duration = speeches[i+1]['start'] - speech['end']
            if silence_duration < 2 * speech_pad_samples:
                speech['end'] += int(silence_duration // 2)
                speeches[i+1]['start'] = int(max(0, speeches[i+1]['start'] - silence_duration // 2))
            else:
                speech['end'] = int(min(audio_length_samples, speech['end'] + speech_pad_samples))
                speeches[i+1]['start'] = int(max(0, speeches[i+1]['start'] - speech_pad_samples))
        else:
            speech['end'] = int(min(audio_length_samples, speech['end'] + speech_pad_samples))

    if return_seconds:
        audio_length_seconds = audio_length_samples / sampling_rate
        for speech_dict in speeches:
            speech_dict['start'] = max(round(speech_dict['start'] / sampling_rate, 1), 0)
            speech_dict['end'] = min(round(speech_dict['end'] / sampling_rate, 1), audio_length_seconds)
    elif step > 1:
        for speech_dict in speeches:
            speech_dict['start'] *= step
            speech_dict['end'] *= step

    if visualize_probs:
        make_visualization(speech_probs, window_size_samples / sampling_rate)

    return speeches


class VADIterator:
    def __init__(self,
                 model,
                 threshold: float = 0.5,
                 sampling_rate: int = 16000,
                 min_silence_duration_ms: int = 100,
                 speech_pad_ms: int = 30
                 ):

        """
        Class for stream imitation

        Parameters
        ----------
        model: preloaded .jit/.onnx silero VAD model

        threshold: float (default - 0.5)
            Speech threshold. Silero VAD outputs speech probabilities for each audio chunk, probabilitie
    s ABOVE this value are considered as SPEECH.
            It is better to tune this parameter for each dataset separately, but "lazy" 0.5 is pretty go
    od for most datasets.

        sampling_rate: int (default - 16000)
```

```
        Currently silero VAD models support 8000 and 16000 sample rates

    min_silence_duration_ms: int (default - 100 milliseconds)
        In the end of each speech chunk wait for min_silence_duration_ms before separating it

    speech_pad_ms: int (default - 30 milliseconds)
        Final speech chunks are padded by speech_pad_ms each side
    """

    self.model = model
    self.threshold = threshold
    self.sampling_rate = sampling_rate

    if sampling_rate not in [8000, 16000]:
        raise ValueError('VADIterator does not support sampling rates other than [8000, 16000]')

    self.min_silence_samples = sampling_rate * min_silence_duration_ms / 1000
    self.speech_pad_samples = sampling_rate * speech_pad_ms / 1000
    self.reset_states()

def reset_states(self):

    self.model.reset_states()
    self.triggered = False
    self.temp_end = 0
    self.current_sample = 0

@torch.no_grad()
def __call__(self, x, return_seconds=False):
    """
    x: torch.Tensor
        audio chunk (see examples in repo)

    return_seconds: bool (default - False)
        whether return timestamps in seconds (default - samples)
    """

    if not torch.is_tensor(x):
        try:
            x = torch.Tensor(x)
        except:
            raise TypeError("Audio cannot be casted to tensor. Cast it manually")

    window_size_samples = len(x[0]) if x.dim() == 2 else len(x)
    self.current_sample += window_size_samples

    speech_prob = self.model(x, self.sampling_rate).item()

    if (speech_prob >= self.threshold) and self.temp_end:
        self.temp_end = 0

    if (speech_prob >= self.threshold) and not self.triggered:
        self.triggered = True
        speech_start = max(0, self.current_sample - self.speech_pad_samples - window_size_samples)
        return {'start': int(speech_start) if not return_seconds else round(speech_start / self.samp
ling_rate, 1)}

    if (speech_prob < self.threshold - 0.15) and self.triggered:
        if not self.temp_end:
            self.temp_end = self.current_sample
```

```
            if self.current_sample - self.temp_end < self.min_silence_samples:
                return None
            else:
                speech_end = self.temp_end + self.speech_pad_samples - window_size_samples
                self.temp_end = 0
                self.triggered = False
                return {'end': int(speech_end) if not return_seconds else round(speech_end / self.sampli
    ng_rate, 1)}

        return None


def collect_chunks(tss: List[dict],
                   wav: torch.Tensor):
    chunks = []
    for i in tss:
        chunks.append(wav[i['start']: i['end']])
    return torch.cat(chunks)


def drop_chunks(tss: List[dict],
                wav: torch.Tensor):
    chunks = []
    cur_start = 0
    for i in tss:
        chunks.append((wav[cur_start: i['start']]))
        cur_start = i['end']
    return torch.cat(chunks)
# ===== 文件: xiaozhi-server/models/snakers4_silero-vad/src/silero_vad/data/__init__.py =====
# ===== 文件: xiaozhi-server/models/SenseVoiceSmall/demo.py =====
from funasr import AutoModel
from funasr.utils.postprocess_utils import rich_transcription_postprocess

model_dir = "./"


model = AutoModel(
    model=model_dir,
    vad_model="fsmn-vad",
    vad_kwargs={"max_single_segment_time": 30000},
    # device="cuda:0",
    hub="hf",
)

# en
res = model.generate(
    input=f"{model.model_path}/example/en.mp3",
    cache={},
    language="auto",  # "zn", "en", "yue", "ja", "ko", "nospeech"
    use_itn=True,
    batch_size_s=60,
    merge_vad=True,  #
    merge_length_s=15,
)
text = rich_transcription_postprocess(res[0]["text"])
print(text)

# ===== 文件: xiaozhi-server/core/connection.py =====
import os
import sys
```

```
import copy
import json
import re
import uuid
import time
import queue
import asyncio
import threading
import traceback
import subprocess
import websockets
import opuslib_next
import numpy as np

from core.utils.util import (
    extract_json_from_string,
    check_vad_update,
    check_asr_update,
    filter_sensitive_info,
)
from typing import Dict, Any
from collections import deque
from core.utils.modules_initialize import (
    initialize_modules,
    initialize_tts,
    initialize_asr,
)
from core.handle.reportHandle import report, enqueue_tool_report
from core.providers.tts.default import DefaultTTS
from concurrent.futures import ThreadPoolExecutor
from core.utils.dialogue import Message, Dialogue

from core.providers.asr.dto.dto import InterfaceType
from core.handle.textHandle import handleTextMessage
from core.providers.tools.unified_tool_handler import UnifiedToolHandler
from plugins_func.loadplugins import auto_import_modules
from plugins_func.register import Action, ActionResponse
from core.auth import AuthenticationError
from config.config_loader import get_private_config_from_api
from core.providers.tts.dto.dto import ContentType, TTSMessageDTO, SentenceType
from config.logger import setup_logging, build_module_string, create_connection_logger
from config.manage_api_client import DeviceNotFoundException, DeviceBindException, generate_and_save_cha
    t_title
from core.utils.prompt_manager import PromptManager
from core.utils.voiceprint_provider import VoiceprintProvider
from core.utils.util import get_system_error_response
from core.utils import textUtils
from core.content_safety import (
    ContentSafetyContext,
    ContentSafetyGate,
    OutputSafetyGate,
)
from core.utils.content_safety import create_content_safety_provider


TAG = __name__

auto_import_modules("plugins_func.functions")
```

```
class TTSException(RuntimeError):
    pass

# direct_answer 虚拟工具定义
# 不是真实工具，是路由机制：将"调不调工具"的二选一变为"调哪个"的多选，防止小模型误触发真实工具
DIRECT_ANSWER_TOOL = {
    "type": "function",
    "function": {
        "name": "direct_answer",
        "description": "当用户的请求不匹配其他任何工具时，可用此选项直接回复。将回复内容写在response参数
    里。",
        "parameters": {
            "type": "object",
            "properties": {
                "response": {
                    "type": "string",
                    "description": "你回复用户的完整内容",
                },
            },
            "required": ["response"],
        },
    },
}


class ConnectionHandler:
    def __init__(
            self,
            config: Dict[str, Any],
            _vad,
            _asr,
            _llm,
            _memory,
            _intent,
            server=None,
            content_safety_provider=None,
    ):
        self.common_config = config
        self.config = copy.deepcopy(config)
        self.session_id = str(uuid.uuid4())
        self.logger = setup_logging()
        self.server = server  # 保存server实例的引用
        self.content_safety_provider = (
            content_safety_provider or create_content_safety_provider(config)
        )
        self.content_safety_gate = ContentSafetyGate(
            self.content_safety_provider,
            self.config,
            audit_log=self.logger.bind(tag=TAG).info,
        )

        self.need_bind = False  # 是否需要绑定设备
        self.bind_completed_event = asyncio.Event()
        self.bind_code = None  # 绑定设备的验证码
        self.last_bind_prompt_time = 0  # 上次播放绑定提示的时间戳(秒)
        self.bind_prompt_interval = 60  # 绑定提示播放间隔(秒)

        self.read_config_from_api = self.config.get("read_config_from_api", False)

        self.websocket: websockets.ServerConnection | None = None
```

```
self.headers = None
self.device_id = None
self.client_ip = None
self.prompt = None
self.welcome_msg = None
self.max_output_size = 0
self.chat_history_conf = 0
self.audio_format = "opus"
self.sample_rate = 24000  # 默认采样率，从客户端 hello 消息中动态更新

# 客户端状态相关
self.client_abort = False
# 当前轮次是否命中危机内容（轻生等）；入口放行时置 True，出口兜底据此分级
self.crisis_context = False
self.client_is_speaking = False
self.client_listen_mode = "auto"
self.client_aec = False  # 是否启用了服务端AEC

# 线程任务相关
self.loop = None  # 在 handle_connection 中获取运行中的事件循环
self.stop_event = threading.Event()
self.executor = ThreadPoolExecutor(max_workers=5)

# 添加上报线程池
self.report_queue = queue.Queue(maxsize=100)  # 限制上报队列大小，防止内存溢出（优化：1000→100）
self.report_thread = None
# 未来可以通过修改此处，调节asr的上报和tts的上报，目前默认都开启
self.report_asr_enable = self.read_config_from_api
self.report_tts_enable = self.read_config_from_api

# 依赖的组件
self.vad = None
self.asr = None
self.tts = None
self._asr = _asr
self._vad = _vad
self.llm = _llm
self.memory = _memory
self.intent = _intent

# 为每个连接单独管理声纹识别
self.voiceprint_provider = None

# vad相关变量
self.client_audio_buffer = bytearray()
self.client_have_voice = False
self.client_voice_window = deque(maxlen=5)
self.first_activity_time = 0.0  # 记录首次活动的时间（毫秒）
self.last_activity_time = 0.0  # 统一的活动时间戳（毫秒）
self.vad_last_voice_time = 0.0  # 记录用户最后一次说话的时间（毫秒）
self.client_voice_stop = False
self.last_is_voice = False

# asr相关变量
# 因为实际部署时可能会用到公共的本地ASR，不能把变量暴露给公共ASR
# 所以涉及到ASR的变量，需要在这里定义，属于connection的私有变量
self.asr_audio = []  # 存储PCM帧列表，供VAD和ASR共享
self.asr_audio_queue = queue.Queue(maxsize=100)  # 限制音频队列大小，约100帧PCM数据
self.current_speaker = None  # 存储当前说话人
self.introduced_speakers = set()  # 已"首次引入"的说话人，控制只在首轮带名字
```

```
    self.system_introduced_speakers = set()  # 已在 system 注入过身份的说话人，控制 system 身份只首
轮出现

    # llm相关变量
    self.dialogue = Dialogue()

    # tts相关变量
    self.sentence_id = None
    # 处理TTS响应没有文本返回
    self.tts_MessageText = ""

    # iot相关变量
    self.iot_descriptors = {}
    self.func_handler = None

    self.cmd_exit = self.config["exit_commands"]

    # 是否在聊天结束后关闭连接
    self.close_after_chat = False
    self.load_function_plugin = False
    self.intent_type = "nointent"

    self.timeout_seconds = (
        int(self.config.get("close_connection_no_voice_time", 120)) + 60
    )  # 在原来第一道关闭的基础上加60秒，进行二道关闭
    self.timeout_task = None
    self._background_init_task = None

    # {"mcp":true} 表示启用MCP功能
    self.features = None

    # 标记连接是否来自MQTT
    self.conn_from_mqtt_gateway = False

    # 初始化提示词管理器
    self.prompt_manager = PromptManager(self.config, self.logger)

    # 初始化通话状态
    self.calling = False
    # 标记当前是否为来电接听模式
    self.incoming_call = None

async def handle_connection(self, ws: websockets.ServerConnection):
    try:
        # 获取运行中的事件循环（必须在异步上下文中）
        self.loop = asyncio.get_running_loop()

        # 获取并验证headers
        self.headers = dict(ws.request.headers)
        real_ip = self.headers.get("x-real-ip") or self.headers.get(
            "x-forwarded-for"
        )
        if real_ip:
            self.client_ip = real_ip.split(",")[0].strip()
        else:
            self.client_ip = ws.remote_address[0]
        self.logger.bind(tag=TAG).info(
            f"{self.client_ip} conn - Headers: {self.headers}"
        )
```

```
        self.device_id = self.headers.get("device-id", None)

        # 认证通过,继续处理
        self.websocket = ws

        # 检查是否来自MQTT连接
        request_path = ws.request.path
        self.conn_from_mqtt_gateway = request_path.endswith("?from=mqtt_gateway")
        if self.conn_from_mqtt_gateway:
            self.logger.bind(tag=TAG).info("连接来自:MQTT网关")

        # 初始化活动时间戳
        self.first_activity_time = time.time() * 1000
        self.last_activity_time = time.time() * 1000

        # 启动超时检查任务
        self.timeout_task = asyncio.create_task(self._check_timeout())

        # 启动AEC缓存清理任务
        self._aec_cache_cleanup_task = asyncio.create_task(self._check_aec_cache_expiry())

        self.welcome_msg = self.config["xiaozhi"]
        self.welcome_msg["session_id"] = self.session_id

        # 从配置中读取采样率
        self.sample_rate = self.welcome_msg["audio_params"]["sample_rate"]
        self.logger.bind(tag=TAG).info(f"配置输出音频采样率为: {self.sample_rate}")

        # 在后台初始化配置和组件（完全不阻塞主循环）
        self._background_init_task = asyncio.create_task(self._background_initialize())

        try:
            async for message in self.websocket:
                await self._route_message(message)
        except websockets.exceptions.ConnectionClosed:
            self.logger.bind(tag=TAG).info("客户端断开连接")

    except AuthenticationError as e:
        self.logger.bind(tag=TAG).error(f"Authentication failed: {str(e)}")
        return
    except Exception as e:
        stack_trace = traceback.format_exc()
        self.logger.bind(tag=TAG).error(f"Connection error: {str(e)}-{stack_trace}")
        return
    finally:
        try:
            await self._save_and_close(ws)
        except Exception as final_error:
            self.logger.bind(tag=TAG).error(f"最终清理时出错: {final_error}")
            # 确保即使保存记忆失败，也要关闭连接
            try:
                await self.close(ws)
            except Exception as close_error:
                self.logger.bind(tag=TAG).error(
                    f"强制关闭连接时出错: {close_error}"
                )

async def _save_and_close(self, ws):
    """保存记忆并关闭连接"""
    self.logger.bind(tag=TAG).info(f"_save_and_close called, memory={self.memory is not None}, dialo
```

```
gue_len={len(self.dialogue.dialogue) if self.dialogue else 0}")
    try:
        # 守护线程1：独立生成标题（不依赖记忆模型）
        if self.session_id:
            def generate_title_task():
                try:
                    loop = asyncio.new_event_loop()
                    asyncio.set_event_loop(loop)
                    loop.run_until_complete(
                        generate_and_save_chat_title(self.session_id)
                    )
                except Exception as e:
                    self.logger.bind(tag=TAG).error(f"生成标题失败: {e}")
                finally:
                    try:
                        loop.close()
                    except Exception:
                        pass

            threading.Thread(target=generate_title_task, daemon=False).start()

        # 守护线程2：异步保存记忆（仅记忆，不含标题）
        if self.memory:
            def save_memory_task():
                try:
                    loop = asyncio.new_event_loop()
                    asyncio.set_event_loop(loop)
                    loop.run_until_complete(
                        self.memory.save_memory(
                            self.dialogue.dialogue, self.session_id
                        )
                    )
                except Exception as e:
                    self.logger.bind(tag=TAG).error(f"保存记忆失败: {e}")
                finally:
                    try:
                        loop.close()
                    except Exception:
                        pass

            threading.Thread(target=save_memory_task, daemon=True).start()
    except Exception as e:
        self.logger.bind(tag=TAG).error(f"保存记忆失败: {e}")
    finally:
        # 关闭连接（记忆保存线程在后台继续执行）
        try:
            await self.close(ws)
        except Exception as close_error:
            self.logger.bind(tag=TAG).error(
                f"保存记忆后关闭连接失败: {close_error}"
            )

async def _discard_message_with_bind_prompt(self):
    """丢弃消息并检查是否需要播放绑定提示"""
    current_time = time.time()
    # 检查是否需要播放绑定提示
    if current_time - self.last_bind_prompt_time >= self.bind_prompt_interval:
        self.last_bind_prompt_time = current_time
        # 复用现有的绑定提示逻辑
        from core.handle.receiveAudioHandle import check_bind_device
```

```
    """将用户输入的中文类别映射到配置文件中的类别键"""
    if not category_text:
        return None

    # 类别映射字典，目前支持社会、国际、财经新闻，如需更多类型，参见配置文件
    category_map = {
        # 社会新闻
        "社会": "society_rss_url",
        "社会新闻": "society_rss_url",
        # 国际新闻
        "国际": "world_rss_url",
        "国际新闻": "world_rss_url",
        # 财经新闻
        "财经": "finance_rss_url",
        "财经新闻": "finance_rss_url",
        "金融": "finance_rss_url",
        "经济": "finance_rss_url",
    }

    # 转换为小写并去除空格
    normalized_category = category_text.lower().strip()

    # 返回映射结果，如果没有匹配项则返回原始输入
    return category_map.get(normalized_category, category_text)


@register_function(
    "get_news_from_chinanews",
    GET_NEWS_FROM_CHINANEWS_FUNCTION_DESC,
    ToolType.SYSTEM_CTL,
)
async def get_news_from_chinanews(
    conn: "ConnectionHandler",
    category: str = None,
    detail: bool = False,
    lang: str = "zh_CN",
):
    """获取新闻并随机选择一条进行播报，或获取上一条新闻的详细内容"""
    try:
        # 如果detail为True，获取上一条新闻的详细内容
        if detail:
            if (
                not hasattr(conn, "last_news_link")
                or not conn.last_news_link
                or "link" not in conn.last_news_link
            ):
                return ActionResponse(
                    Action.REQLLM,
                    "抱歉，没有找到最近查询的新闻，请先获取一条新闻。",
                    None,
                )

            link = conn.last_news_link.get("link")
            title = conn.last_news_link.get("title", "未知标题")

            if link == "#":
                return ActionResponse(
                    Action.REQLLM, "抱歉，该新闻没有可用的链接获取详细内容。", None
                )
```

```
    logger.bind(tag=TAG).debug(f"获取新闻详情: {title}, URL={link}")

    # 获取新闻详情
    detail_content = await fetch_news_detail(link)

    if not detail_content or detail_content == "无法获取详细内容":
        return ActionResponse(
            Action.REQLLM,
            f"抱歉，无法获取《{title}》的详细内容，可能是链接已失效或网站结构发生变化。",
            None,
        )

    # 构建详情报告
    detail_report = (
        f"根据下列数据，用{lang}回应用户的新闻详情查询请求：\n\n"
        f"新闻标题: {title}\n"
        f"详细内容: {detail_content}\n\n"
        f"(请对上述新闻内容进行总结，提取关键信息，以自然、流畅的方式向用户播报，"
        f"不要提及这是总结，就像是在讲述一个完整的新闻故事)"
    )

    return ActionResponse(Action.REQLLM, detail_report, None)

# 否则，获取新闻列表并随机选择一条
# 从配置中获取RSS URL
rss_config = conn.config.get("plugins", {}).get("get_news_from_chinanews", {})
default_rss_url = rss_config.get(
    "default_rss_url", "https://www.chinanews.com.cn/rss/society.xml"
)

# 将用户输入的类别映射到配置中的类别键
mapped_category = map_category(category)

# 如果提供了类别，尝试从配置中获取对应的URL
rss_url = default_rss_url
if mapped_category and mapped_category in rss_config:
    rss_url = rss_config[mapped_category]

logger.bind(tag=TAG).info(
    f"获取新闻: 原始类别={category}, 映射类别={mapped_category}, URL={rss_url}"
)

# 获取新闻列表
news_items = await fetch_news_from_rss(rss_url)

if not news_items:
    return ActionResponse(
        Action.REQLLM, "抱歉，未能获取到新闻信息，请稍后再试。", None
    )

# 随机选择一条新闻
selected_news = random.choice(news_items)

# 保存当前新闻链接到连接对象，以便后续查询详情
if not hasattr(conn, "last_news_link"):
    conn.last_news_link = {}
conn.last_news_link = {
    "link": selected_news.get("link", "#"),
    "title": selected_news.get("title", "未知标题"),
}
```

```
        # 构建新闻报告
        news_report = (
            f"根据下列数据，用{lang}回应用户的新闻查询请求：\n\n"
            f"新闻标题: {selected_news['title']}\n"
            f"发布时间: {selected_news['pubDate']}\n"
            f"新闻内容: {selected_news['description']}\n"
            f"(请以自然、流畅的方式向用户播报这条新闻，可以适当总结内容，"
            f"直接读出新闻即可，不需要额外多余的内容。"
            f"如果用户询问更多详情，告知用户可以说'请详细介绍这条新闻'获取更多内容)"
        )

        return ActionResponse(Action.REQLLM, news_report, None)

    except Exception as e:
        logger.bind(tag=TAG).error(f"获取新闻出错: {e}")
        return ActionResponse(
            Action.REQLLM, "抱歉，获取新闻时发生错误，请稍后再试。", None
        )
# ===== 文件: xiaozhi-server/plugins_func/functions/get_news_from_newsnow.py =====
import random
import httpx
from io import BytesIO
from markitdown import MarkItDown, StreamInfo
from config.logger import setup_logging
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler


TAG = __name__
logger = setup_logging()

CHANNEL_MAP = {
    "V2EX": "v2ex-share",
    "知乎": "zhihu",
    "微博": "weibo",
    "联合早报": "zaobao",
    "酷安": "coolapk",
    "MKTNews": "mktnews-flash",
    "华尔街见闻": "wallstreetcn-quick",
    "36氪": "36kr-quick",
    "抖音": "douyin",
    "虎扑": "hupu",
    "百度贴吧": "tieba",
    "今日头条": "toutiao",
    "IT之家": "ithome",
    "澎湃新闻": "thepaper",
    "卫星通讯社": "sputniknewscn",
    "参考消息": "cankaoxiaoxi",
    "远景论坛": "pcbeta-windows11",
    "财联社": "cls-depth",
    "雪球": "xueqiu-hotstock",
    "格隆汇": "gelonghui",
    "法布财经": "fastbull-express",
    "Solidot": "solidot",
    "Hacker News": "hackernews",
    "Product Hunt": "producthunt",
```

```
    "Github": "github-trending-today",
    "哔哩哔哩": "bilibili-hot-search",
    "快手": "kuaishou",
    "靠谱新闻": "kaopu",
    "金十数据": "jin10",
    "百度热搜": "baidu",
    "牛客": "nowcoder",
    "少数派": "sspai",
    "稀土掘金": "juejin",
    "凤凰网": "ifeng",
    "虫部落": "chongbuluo-latest",
}

# 默认新闻来源字典，当配置中没有指定时使用
DEFAULT_NEWS_SOURCES = "澎湃新闻;百度热搜;财联社"

def _get_newsnow_config(conn):
    # 从连接配置获取
    plugins = conn.config.get("plugins", {})
    newsnow = plugins.get("get_news_from_newsnow", {})
    sources = newsnow.get("news_sources", "")
    if isinstance(sources, str) and sources.strip():
        return sources

    return ""

def get_news_sources_from_config(conn):
    """从配置中获取新闻源字符串"""
    try:
        result = _get_newsnow_config(conn)
        if result:
            logger.bind(tag=TAG).debug(f"使用配置的新闻源: {result}")
            return result

        logger.bind(tag=TAG).debug("未找到新闻源配置，使用默认配置")
        return DEFAULT_NEWS_SOURCES

    except Exception as e:
        logger.bind(tag=TAG).error(f"获取新闻源配置失败: {e}，使用默认配置")
        return DEFAULT_NEWS_SOURCES


# 从默认配置获取可用的新闻源名称（运行时由get_news_sources_from_config动态获取）
example_sources_str = DEFAULT_NEWS_SOURCES.replace(";","、")

GET_NEWS_FROM_NEWSNOW_FUNCTION_DESC = {
    "type": "function",
    "function": {
        "name": "get_news_from_newsnow",
        "description": "当用户要求查看或收听新闻时调用（如'来条新闻"今天有什么新闻'）。",
        "parameters": {
            "type": "object",
            "properties": {
                "source": {
                    "type": "string",
                    "description": f"新闻源的标准中文名称，例如{example_sources_str}等。可选参数，如果不
    提供则使用默认新闻源",
                },
                "detail": {
                    "type": "boolean",
```

```
                "description": "是否获取详细内容，默认为false。如果为true，则获取上一条新闻的详细内
    容",
            },
            "lang": {
                "type": "string",
                "description": "返回用户使用的语言code，例如zh_CN/zh_HK/en_US/ja_JP等，默认zh_CN",
            },
        },
        "required": ["lang"],
    },
  },
}


async def fetch_news_from_api(conn: "ConnectionHandler", source="thepaper"):
    """从API获取新闻列表"""
    try:
        api_url = f"https://newsnow.busiyi.world/api/s?id={source}"

        news_config = conn.config.get("plugins", {}).get("get_news_from_newsnow", {})
        if news_config.get("url"):
            api_url = news_config["url"] + source

        headers = {"User-Agent": "Mozilla/5.0"}
        async with httpx.AsyncClient(timeout=httpx.Timeout(10.0, connect=3.0)) as client:
            response = await client.get(api_url, headers=headers)

        data = response.json()

        if "items" in data:
            return data["items"]
        else:
            logger.bind(tag=TAG).error(f"获取新闻API响应格式错误: {data}")
            return []

    except Exception as e:
        logger.bind(tag=TAG).error(f"获取新闻API失败: {e}")
        return []


async def fetch_news_detail(url):
    """获取新闻详情页内容并使用MarkItDown清理HTML"""
    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        async with httpx.AsyncClient(timeout=httpx.Timeout(10.0, connect=3.0)) as client:
            response = await client.get(url, headers=headers)

        # 使用MarkItDown清理HTML内容
        md = MarkItDown(enable_plugins=False)
        result = md.convert_stream(
            BytesIO(response.content),
            stream_info=StreamInfo(
                mimetype="text/html",
                extension=".html",
                charset=response.encoding or "utf-8",
            ),
        )

        # 获取清理后的文本内容
        clean_text = result.text_content
```

```
        # 如果清理后的内容为空，返回提示信息
        if not clean_text or len(clean_text.strip()) == 0:
            logger.bind(tag=TAG).warning(f"清理后的新闻内容为空: {url}")
            return "无法解析新闻详情内容，可能是网站结构特殊或内容受限。"

        return clean_text
    except Exception as e:
        logger.bind(tag=TAG).error(f"获取新闻详情失败: {e}")
        return "无法获取详细内容"


@register_function(
    "get_news_from_newsnow",
    GET_NEWS_FROM_NEWSNOW_FUNCTION_DESC,
    ToolType.SYSTEM_CTL,
)
async def get_news_from_newsnow(
    conn: "ConnectionHandler",
    source: str = "澎湃新闻",
    detail: bool = False,
    lang: str = "zh_CN",
):
    """获取新闻并随机选择一条进行播报，或获取上一条新闻的详细内容"""
    try:
        # 获取当前配置的新闻源
        news_sources = get_news_sources_from_config(conn)

        # 如果detail为True，获取上一条新闻的详细内容
        detail = str(detail).lower() == "true"
        if detail:
            if (
                not hasattr(conn, "last_newsnow_link")
                or not conn.last_newsnow_link
                or "url" not in conn.last_newsnow_link
            ):
                return ActionResponse(
                    Action.REQLLM,
                    "抱歉，没有找到最近查询的新闻，请先获取一条新闻。",
                    None,
                )

            url = conn.last_newsnow_link.get("url")
            title = conn.last_newsnow_link.get("title", "未知标题")
            source_id = conn.last_newsnow_link.get("source_id", "thepaper")
            source_name = CHANNEL_MAP.get(source_id, "未知来源")

            if not url or url == "#":
                return ActionResponse(
                    Action.REQLLM, "抱歉，该新闻没有可用的链接获取详细内容。", None
                )

            logger.bind(tag=TAG).debug(
                f"获取新闻详情: {title}, 来源: {source_name}, URL={url}"
            )

            # 获取新闻详情
            detail_content = await fetch_news_detail(url)

            if not detail_content or detail_content == "无法获取详细内容":
```

```
            return ActionResponse(
                Action.REQLLM,
                f"抱歉，无法获取《{title}》的详细内容，可能是链接已失效或网站结构发生变化。",
                None,
            )

        # 构建详情报告
        detail_report = (
            f"根据下列数据，用{lang}回应用户的新闻详情查询请求：\n\n"
            f"新闻标题: {title}\n"
            # f"新闻来源: {source_name}\n"
            f"详细内容: {detail_content}\n\n"
            f"(请对上述新闻内容进行总结，提取关键信息，以自然、流畅的方式向用户播报，"
            f"不要提及这是总结，就像是在讲述一个完整的新闻)"
        )

        return ActionResponse(Action.REQLLM, detail_report, None)

    # 否则，获取新闻列表并随机选择一条
    # 将中文名称转换为英文ID
    english_source_id = None

    # 检查输入的中文名称是否在配置的新闻源中
    news_sources_list = [
        name.strip() for name in news_sources.split(";") if name.strip()
    ]
    if source in news_sources_list:
        # 如果输入的中文名称在配置的新闻源中，在 CHANNEL_MAP 中查找对应的英文ID
        english_source_id = CHANNEL_MAP.get(source)

    # 如果找不到对应的英文ID，使用默认源
    if not english_source_id:
        logger.bind(tag=TAG).warning(f"无效的新闻源: {source}，使用默认源澎湃新闻")
        english_source_id = "thepaper"
        source = "澎湃新闻"

    logger.bind(tag=TAG).info(f"获取新闻: 新闻源={source}({english_source_id})")

    # 获取新闻列表
    news_items = await fetch_news_from_api(conn, english_source_id)

    if not news_items:
        return ActionResponse(
            Action.REQLLM,
            f"抱歉，未能从{source}获取到新闻信息，请稍后再试或尝试其他新闻源。",
            None,
        )

    # 随机选择一条新闻
    selected_news = random.choice(news_items)

    # 保存当前新闻链接到连接对象，以便后续查询详情
    if not hasattr(conn, "last_newsnow_link"):
        conn.last_newsnow_link = {}
    conn.last_newsnow_link = {
        "url": selected_news.get("url", "#"),
        "title": selected_news.get("title", "未知标题"),
        "source_id": english_source_id,
    }
```

```
        # 构建新闻报告
        news_report = (
            f"根据下列数据，用{lang}回应用户的新闻查询请求：\n\n"
            f"新闻标题: {selected_news['title']}\n"
            # f"新闻来源: {source}\n"
            f"(请以自然、流畅的方式向用户播报这条新闻标题，"
            f"提示用户可以要求获取详细内容，此时会获取新闻的详细内容。)"
        )

        return ActionResponse(Action.REQLLM, news_report, None)

    except Exception as e:
        logger.bind(tag=TAG).error(f"获取新闻出错: {e}")
        return ActionResponse(
            Action.REQLLM, "抱歉，获取新闻时发生错误，请稍后再试。", None
        )
# ===== 文件: xiaozhi-server/plugins_func/functions/get_time.py =====
from datetime import datetime
import cnlunar
from plugins_func.register import register_function, ToolType, ActionResponse, Action

get_lunar_function_desc = {
    "type": "function",
    "function": {
        "name": "get_lunar",
        "description": (
            "用于具体日期的阴历/农历和黄历信息。"
            "用户可以指定查询内容，如：阴历日期、天干地支、节气、生肖、星座、八字、宜忌等。"
            "如果没有指定查询内容，则默认查询干支年和农历日期。"
            "对于'今天农历是多少'、'今天农历日期'这样的基本查询，请直接使用context中的信息，不要调用此工
    具。"
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "date": {
                    "type": "string",
                    "description": "要查询的日期，格式为YYYY-MM-DD，例如2024-01-01。如果不提供，则使用当
    前日期",
                },
                "query": {
                    "type": "string",
                    "description": "要查询的内容，例如阴历日期、天干地支、节日、节气、生肖、星座、八字、
    宜忌等",
                },
            },
            "required": [],
        },
    },
}


@register_function("get_lunar", get_lunar_function_desc, ToolType.WAIT)
def get_lunar(date=None, query=None):
    """
    用于获取当前的阴历/农历，和天干地支、节气、生肖、星座、八字、宜忌等黄历信息
    """
    from core.utils.cache.manager import cache_manager, CacheType

    # 如果提供了日期参数，则使用指定日期；否则使用当前日期
```

```
if date:
    try:
        now = datetime.strptime(date, "%Y-%m-%d")
    except ValueError:
        return ActionResponse(
            Action.REQLLM,
            f"日期格式错误，请使用YYYY-MM-DD格式，例如：2024-01-01",
            None,
        )
else:
    now = datetime.now()

current_date = now.strftime("%Y-%m-%d")

# 如果 query 为 None，则使用默认文本
if query is None:
    query = "默认查询干支年和农历日期"

# 尝试从缓存获取农历信息
lunar_cache_key = f"lunar_info_{current_date}"
cached_lunar_info = cache_manager.get(CacheType.LUNAR, lunar_cache_key)
if cached_lunar_info:
    return ActionResponse(Action.REQLLM, cached_lunar_info, None)

response_text = f"根据以下信息回应用户的查询请求，并提供与{query}相关的信息：\n"

lunar = cnlunar.Lunar(now, godType="8char")
response_text += (
    "农历信息：\n"
    "%s年%s%s\n" % (lunar.lunarYearCn, lunar.lunarMonthCn[:-1], lunar.lunarDayCn)
    + "干支: %s年 %s月 %s日\n" % (lunar.year8Char, lunar.month8Char, lunar.day8Char)
    + "生肖: 属%s\n" % (lunar.chineseYearZodiac)
    + "八字: %s\n"
    % (
        " ".join(
            [lunar.year8Char, lunar.month8Char, lunar.day8Char, lunar.twohour8Char]
        )
    )
    + "今日节日: %s\n"
    % (
        ",".join(
            filter(
                None,
                (
                    lunar.get_legalHolidays(),
                    lunar.get_otherHolidays(),
                    lunar.get_otherLunarHolidays(),
                ),
            )
        )
    )
    + "今日节气: %s\n" % (lunar.todaySolarTerms)
    + "下一节气: %s %s年%s月%s日\n"
    % (
        lunar.nextSolarTerm,
        lunar.nextSolarTermYear,
        lunar.nextSolarTermDate[0],
        lunar.nextSolarTermDate[1],
    )
    + "今年节气表: %s\n"
```

```
        % (
            ", ".join(
                [
                    f"{term}({date[0]}月{date[1]}日)"
                    for term, date in lunar.thisYearSolarTermsDic.items()
                ]
            )
        )
        + "生肖冲煞: %s\n" % (lunar.chineseZodiacClash)
        + "星座: %s\n" % (lunar.starZodiac)
        + "纳音: %s\n" % lunar.get_nayin()
        + "彭祖百忌: %s\n" % (lunar.get_pengTaboo(delimit=", "))
        + "值日: %s执位\n" % lunar.get_today12DayOfficer()[0]
        + "值神: %s(%s)\n"
        % (lunar.get_today12DayOfficer()[1], lunar.get_today12DayOfficer()[2])
        + "廿八宿: %s\n" % lunar.get_the28Stars()
        + "吉神方位: %s\n" % " ".join(lunar.get_luckyGodsDirection())
        + "今日胎神: %s\n" % lunar.get_fetalGod()
        + "宜: %s\n" % "、".join(lunar.goodThing[:10])
        + "忌: %s\n" % "、".join(lunar.badThing[:10])
        + "(默认返回干支年和农历日期；仅在要求查询宜忌信息时才返回本日宜忌)"
    )

    # 缓存农历信息
    cache_manager.set(CacheType.LUNAR, lunar_cache_key, response_text)

    return ActionResponse(Action.REQLLM, response_text, None)
# ===== 文件: xiaozhi-server/plugins_func/functions/get_weather.py =====
import httpx
from bs4 import BeautifulSoup
from config.logger import setup_logging
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from core.utils.util import get_ip_info
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

GET_WEATHER_FUNCTION_DESC = {
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": (
            "获取某个地点的天气，用户应提供一个位置，比如用户说杭州天气，参数为：杭州。"
            "如果用户说的是省份，默认用省会城市。如果用户说的不是省份或城市而是一个地名，默认用该地所在省份的省会城市。"
            "重要：本地未来7天天气已在上下文中提供，用户未指明其他城市时绝对不要调用此工具。"
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "地点名，例如杭州。可选参数，如果不提供则不传",
                },
                "lang": {
                    "type": "string",
```

```
                "description": "返回用户使用的语言code，例如zh_CN/zh_HK/en_US/ja_JP等，默认zh_CN",
            },
        },
        "required": ["lang"],
    },
  },
}

HEADERS = {
  "User-Agent": (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/92.0.4515.107 Safari/537.36"
  )
}

# 天气代码 https://dev.qweather.com/docs/resource/icons/#weather-icons
WEATHER_CODE_MAP = {
  "100": "晴",
  "101": "多云",
  "102": "少云",
  "103": "晴间多云",
  "104": "阴",
  "150": "晴",
  "151": "多云",
  "152": "少云",
  "153": "晴间多云",
  "300": "阵雨",
  "301": "强阵雨",
  "302": "雷阵雨",
  "303": "强雷阵雨",
  "304": "雷阵雨伴有冰雹",
  "305": "小雨",
  "306": "中雨",
  "307": "大雨",
  "308": "极端降雨",
  "309": "毛毛雨/细雨",
  "310": "暴雨",
  "311": "大暴雨",
  "312": "特大暴雨",
  "313": "冻雨",
  "314": "小到中雨",
  "315": "中到大雨",
  "316": "大到暴雨",
  "317": "暴雨到大暴雨",
  "318": "大暴雨到特大暴雨",
  "350": "阵雨",
  "351": "强阵雨",
  "399": "雨",
  "400": "小雪",
  "401": "中雪",
  "402": "大雪",
  "403": "暴雪",
  "404": "雨夹雪",
  "405": "雨雪天气",
  "406": "阵雨夹雪",
  "407": "阵雪",
  "408": "小到中雪",
  "409": "中到大雪",
  "410": "大到暴雪",
  "456": "阵雨夹雪",
```

```
    "457": "阵雪",
    "499": "雪",
    "500": "薄雾",
    "501": "雾",
    "502": "霾",
    "503": "扬沙",
    "504": "浮尘",
    "507": "沙尘暴",
    "508": "强沙尘暴",
    "509": "浓雾",
    "510": "强浓雾",
    "511": "中度霾",
    "512": "重度霾",
    "513": "严重霾",
    "514": "大雾",
    "515": "特强浓雾",
    "900": "热",
    "901": "冷",
    "999": "未知",
}


async def fetch_city_info(location, api_key, api_host):
    url = f"https://{api_host}/geo/v2/city/lookup?key={api_key}&location={location}&lang=zh"
    async with httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=3.0)) as client:
        response = await client.get(url, headers=HEADERS)
    data = response.json()
    if data.get("error") is not None:
        logger.bind(tag=TAG).error(
            f"获取天气失败，原因：{data.get('error', {}).get('detail')}"
        )
        return None
    return data.get("location", [])[0] if data.get("location") else None


async def fetch_weather_page(url):
    async with httpx.AsyncClient(timeout=httpx.Timeout(10.0, connect=3.0)) as client:
        response = await client.get(url, headers=HEADERS)
    return BeautifulSoup(response.text, "html.parser") if response.status_code == 200 else None


def parse_weather_info(soup):
    city_name = soup.select_one("h1.c-submenu__location").get_text(strip=True)

    current_abstract = soup.select_one(".c-city-weather-current .current-abstract")
    current_abstract = (
        current_abstract.get_text(strip=True) if current_abstract else "未知"
    )

    current_basic = {}
    for item in soup.select(
        ".c-city-weather-current .current-basic .current-basic___item"
    ):
        parts = item.get_text(strip=True, separator=" ").split(" ")
        if len(parts) == 2:
            key, value = parts[1], parts[0]
            current_basic[key] = value

    temps_list = []
    for row in soup.select(".city-forecast-tabs__row")[:7]:  # 取前7天的数据
```

```
        date = row.select_one(".date-bg .date").get_text(strip=True)
        weather_code = (
            row.select_one(".date-bg .icon")["src"].split("/")[-1].split(".")[0]
        )
        weather = WEATHER_CODE_MAP.get(weather_code, "未知")
        temps = [span.get_text(strip=True) for span in row.select(".tmp-cont .temp")]
        high_temp, low_temp = (temps[0], temps[-1]) if len(temps) >= 2 else (None, None)
        temps_list.append((date, weather, high_temp, low_temp))

    return city_name, current_abstract, current_basic, temps_list


@register_function("get_weather", GET_WEATHER_FUNCTION_DESC, ToolType.SYSTEM_CTL)
async def get_weather(conn: "ConnectionHandler", location: str = None, lang: str = "zh_CN"):
    from core.utils.cache.manager import cache_manager, CacheType

    weather_config = conn.config.get("plugins", {}).get("get_weather", {})
    api_host = weather_config.get("api_host", "mj7p3y7naa.re.qweatherapi.com")
    api_key = weather_config.get("api_key", "a861d0d5e7bf4ee1a83d9a9e4f96d4da")
    default_location = weather_config.get("default_location", "广州")
    client_ip = conn.client_ip

    # 优先使用用户提供的location参数
    if not location:
        # 通过客户端IP解析城市
        if client_ip:
            # 先从缓存获取IP对应的城市信息
            cached_ip_info = cache_manager.get(CacheType.IP_INFO, client_ip)
            if cached_ip_info:
                location = cached_ip_info.get("city")
            else:
                # 缓存未命中，调用API获取
                ip_info = get_ip_info(client_ip, logger)
                if ip_info:
                    cache_manager.set(CacheType.IP_INFO, client_ip, ip_info)
                    location = ip_info.get("city")

            if not location:
                location = default_location
        else:
            # 若无IP，使用默认位置
            location = default_location
    # 尝试从缓存获取完整天气报告
    weather_cache_key = f"full_weather_{location}_{lang}"
    cached_weather_report = cache_manager.get(CacheType.WEATHER, weather_cache_key)
    if cached_weather_report:
        return ActionResponse(Action.REQLLM, cached_weather_report, None)

    # 缓存未命中，获取实时天气数据
    city_info = await fetch_city_info(location, api_key, api_host)
    if not city_info:
        return ActionResponse(
            Action.REQLLM, f"未找到相关的城市: {location}，请确认地点是否正确", None
        )
    soup = await fetch_weather_page(city_info["fxLink"])
    if not soup:
        return ActionResponse(Action.REQLLM, None, "请求失败")
    city_name, current_abstract, current_basic, temps_list = parse_weather_info(soup)

    weather_report = f"您查询的位置是：{city_name}\n\n当前天气: {current_abstract}\n"
```

```
    # 添加有效的当前天气参数
    if current_basic:
        weather_report += "详细参数：\n"
        for key, value in current_basic.items():
            if value != "0": # 过滤无效值
                weather_report += f"  · {key}: {value}\n"

    # 添加7天预报
    weather_report += "\n未来7天预报：\n"
    for date, weather, high, low in temps_list:
        weather_report += f"{date}: {weather}，气温 {low}~{high}\n"

    # 提示语
    weather_report += "\n（如需某一天的具体天气，请告诉我日期）"

    # 缓存完整的天气报告
    cache_manager.set(CacheType.WEATHER, weather_cache_key, weather_report)

    return ActionResponse(Action.REQLLM, weather_report, None)
# ===== 文件: xiaozhi-server/plugins_func/functions/handle_exit_intent.py =====
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from config.logger import setup_logging
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

handle_exit_intent_function_desc = {
    "type": "function",
    "function": {
        "name": "handle_exit_intent",
        "description": "当用户想结束对话或需要退出系统时调用",
        "parameters": {
            "type": "object",
            "properties": {
                "say_goodbye": {
                    "type": "string",
                    "description": "和用户友好结束对话的告别语",
                }
            },
            "required": ["say_goodbye"],
        },
    },
}


@register_function(
    "handle_exit_intent", handle_exit_intent_function_desc, ToolType.SYSTEM_CTL
)
def handle_exit_intent(conn: "ConnectionHandler", say_goodbye: str | None = None):
    # 处理退出意图
    try:
        if say_goodbye is None:
            say_goodbye = "再见，祝您生活愉快！"
        if not conn.close_after_chat:
            conn.close_after_chat = True
```

```
        logger.bind(tag=TAG).info(f"退出意图已处理:{say_goodbye}")
        return ActionResponse(
            action=Action.RESPONSE, result="退出意图已处理", response=say_goodbye
        )
    except Exception as e:
        logger.bind(tag=TAG).error(f"处理退出意图错误: {e}")
        return ActionResponse(
            action=Action.NONE, result="退出意图处理失败", response=""
        )
# ===== 文件: xiaozhi-server/plugins_func/functions/hass_get_state.py =====
import httpx
from config.logger import setup_logging
from plugins_func.functions.hass_init import initialize_hass_handler
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

hass_get_state_function_desc = {
    "type": "function",
    "function": {
        "name": "hass_get_state",
        "description": "获取homeassistant里设备的状态,包括查询灯光亮度、颜色、色温,媒体播放器的音量,设备
    的暂停、继续操作",
        "parameters": {
            "type": "object",
            "properties": {
                "entity_id": {
                    "type": "string",
                    "description": "需要操作的设备id,homeassistant里的entity_id",
                }
            },
            "required": ["entity_id"],
        },
    },
}


@register_function("hass_get_state", hass_get_state_function_desc, ToolType.SYSTEM_CTL)
async def hass_get_state(conn: "ConnectionHandler", entity_id=""):
    try:
        ha_response = await handle_hass_get_state(conn, entity_id)
        return ActionResponse(Action.REQLLM, ha_response, None)
    except httpx.TimeoutException:
        logger.bind(tag=TAG).error("获取Home Assistant状态超时")
        return ActionResponse(Action.ERROR, "请求超时", None)
    except Exception as e:
        error_msg = f"执行Home Assistant操作失败"
        logger.bind(tag=TAG).error(error_msg)
        return ActionResponse(Action.ERROR, error_msg, None)


async def handle_hass_get_state(conn: "ConnectionHandler", entity_id):
    ha_config = initialize_hass_handler(conn)
    api_key = ha_config.get("api_key")
    base_url = ha_config.get("base_url")
```

```
    url = f"{base_url}/api/states/{entity_id}"
    headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}

    async with httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=3.0)) as client:
        response = await client.get(url, headers=headers)

    if response.status_code == 200:
        responsetext = "设备状态:" + response.json()["state"] + " "
        logger.bind(tag=TAG).info(f"api返回内容: {response.json()}")

        if "media_title" in response.json()["attributes"]:
            responsetext = (
                responsetext
                + "正在播放的是:"
                + str(response.json()["attributes"]["media_title"])
                + " "
            )
        if "volume_level" in response.json()["attributes"]:
            responsetext = (
                responsetext
                + "音量是:"
                + str(response.json()["attributes"]["volume_level"])
                + " "
            )
        if "color_temp_kelvin" in response.json()["attributes"]:
            responsetext = (
                responsetext
                + "色温是:"
                + str(response.json()["attributes"]["color_temp_kelvin"])
                + " "
            )
        if "rgb_color" in response.json()["attributes"]:
            responsetext = (
                responsetext
                + "rgb颜色是:"
                + str(response.json()["attributes"]["rgb_color"])
                + " "
            )
        if "brightness" in response.json()["attributes"]:
            responsetext = (
                responsetext
                + "亮度是:"
                + str(response.json()["attributes"]["brightness"])
                + " "
            )
        logger.bind(tag=TAG).info(f"查询返回内容: {responsetext}")
        return responsetext
    else:
        return f"切换失败，错误码: {response.status_code}"
# ===== 文件: xiaozhi-server/plugins_func/functions/hass_init.py =====
from config.logger import setup_logging
from core.utils.util import check_model_key

TAG = __name__
logger = setup_logging()


def append_devices_to_prompt(conn):
    if conn.intent_type == "function_call":
        funcs = conn.config["Intent"][conn.config["selected_module"]["Intent"]].get(
```

```
            "functions", []
        )

        # 安全地获取插件配置
        plugins_config = conn.config.get("plugins", {})
        config_source = (
            "home_assistant"
            if plugins_config.get("home_assistant")
            else "hass_get_state"
        )

        if "hass_get_state" in funcs or "hass_set_state" in funcs:
            prompt = "\n下面是我家智能设备列表（位置，设备名，entity_id），可以通过homeassistant控制\n"
            deviceStr = plugins_config.get(config_source, {}).get("devices", "")
            conn.prompt += prompt + deviceStr + "\n"
            # 更新提示词
            conn.dialogue.update_system_message(conn.prompt)


def initialize_hass_handler(conn):
    ha_config = {}
    if not conn.load_function_plugin:
        return ha_config

    # 安全地获取插件配置
    plugins_config = conn.config.get("plugins", {})
    # 确定配置来源
    config_source = (
        "home_assistant" if plugins_config.get("home_assistant") else "hass_get_state"
    )
    if not plugins_config.get(config_source):
        return ha_config

    # 统一获取配置
    plugin_config = plugins_config[config_source]
    ha_config["base_url"] = plugin_config.get("base_url")
    ha_config["api_key"] = plugin_config.get("api_key")

    # 统一检查API密钥
    model_key_msg = check_model_key("home_assistant", ha_config.get("api_key"))
    if model_key_msg:
        logger.bind(tag=TAG).error(model_key_msg)

    return ha_config
# ===== 文件: xiaozhi-server/plugins_func/functions/hass_play_music.py =====
import httpx
from config.logger import setup_logging
from plugins_func.functions.hass_init import initialize_hass_handler
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

hass_play_music_function_desc = {
    "type": "function",
    "function": {
```

```
        "name": "hass_play_music",
        "description": "用户想听音乐、有声书的时候使用，在房间的媒体播放器（media_player）里播放对应音频
    ",
        "parameters": {
            "type": "object",
            "properties": {
                "media_content_id": {
                    "type": "string",
                    "description": "可以是音乐或有声书的专辑名称、歌曲名、演唱者,如果未指定就填random",
                },
                "entity_id": {
                    "type": "string",
                    "description": "需要操作的音箱的设备id,homeassistant里的entity_id,media_player开头",
                },
            },
            "required": ["media_content_id", "entity_id"],
        },
    },
}


@register_function(
    "hass_play_music", hass_play_music_function_desc, ToolType.SYSTEM_CTL
)
async def hass_play_music(conn: "ConnectionHandler", entity_id="", media_content_id="random"):
    try:
        result = await handle_hass_play_music(conn, entity_id, media_content_id)
        return ActionResponse(
            action=Action.RECORD, result="指令已接收", response=result
        )
    except Exception as e:
        logger.bind(tag=TAG).error(f"处理音乐意图错误: {e}")
        return ActionResponse(
            action=Action.RESPONSE, result=str(e), response="播放音乐时出错了"
        )


async def handle_hass_play_music(
    conn: "ConnectionHandler", entity_id, media_content_id
):
    ha_config = initialize_hass_handler(conn)
    api_key = ha_config.get("api_key")
    base_url = ha_config.get("base_url")
    url = f"{base_url}/api/services/music_assistant/play_media"
    headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}
    data = {"entity_id": entity_id, "media_id": media_content_id}

    async with httpx.AsyncClient(timeout=httpx.Timeout(10.0, connect=3.0)) as client:
        response = await client.post(url, headers=headers, json=data)

    if response.status_code == 200:
        return f"正在播放{media_content_id}的音乐"
    else:
        return f"音乐播放失败，错误码: {response.status_code}"
# ===== 文件: xiaozhi-server/plugins_func/functions/hass_set_state.py =====
import httpx
from config.logger import setup_logging
from plugins_func.functions.hass_init import initialize_hass_handler
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from typing import TYPE_CHECKING
```

```
if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

hass_set_state_function_desc = {
    "type": "function",
    "function": {
        "name": "hass_set_state",
        "description": "设置homeassistant里设备的状态,包括开、关,调整灯光亮度、颜色、色温,调整播放器的音
    量,设备的暂停、继续、静音操作",
        "parameters": {
            "type": "object",
            "properties": {
                "state": {
                    "type": "object",
                    "properties": {
                        "type": {
                            "type": "string",
                            "description": "需要操作的动作,打开设备:turn_on,关闭设备:turn_off,增加亮度:b
    rightness_up,降低亮度:brightness_down,设置亮度:brightness_value,增加音量:volume_up,降低音量:volume_d
    own,设置音量:volume_set,设置色温:set_kelvin,设置颜色:set_color,设备暂停:pause,设备继续:continue,静音
    /取消静音:volume_mute",
                        },
                        "input": {
                            "type": "integer",
                            "description": "只有在设置音量,设置亮度时候才需要,有效值为1-100,对应音量和亮
    度的1%-100%",
                        },
                        "is_muted": {
                            "type": "string",
                            "description": "只有在设置静音操作时才需要,设置静音的时候该值为true,取消静音
    时该值为false",
                        },
                        "rgb_color": {
                            "type": "array",
                            "items": {"type": "integer"},
                            "description": "只有在设置颜色时需要,这里填目标颜色的rgb值",
                        },
                    },
                    "required": ["type"],
                },
                "entity_id": {
                    "type": "string",
                    "description": "需要操作的设备id,homeassistant里的entity_id",
                },
            },
            "required": ["state", "entity_id"],
        },
    },
}


@register_function("hass_set_state", hass_set_state_function_desc, ToolType.SYSTEM_CTL)
async def hass_set_state(conn: "ConnectionHandler", entity_id="", state=None):
    if state is None:
        state = {}
    try:
```

```
        ha_response = await handle_hass_set_state(conn, entity_id, state)
        return ActionResponse(Action.REQLLM, ha_response, None)
    except httpx.TimeoutException:
        logger.bind(tag=TAG).error("设置Home Assistant状态超时")
        return ActionResponse(Action.ERROR, "请求超时", None)
    except Exception as e:
        error_msg = f"执行Home Assistant操作失败"
        logger.bind(tag=TAG).error(error_msg)
        return ActionResponse(Action.ERROR, error_msg, None)


async def handle_hass_set_state(conn: "ConnectionHandler", entity_id, state):
    ha_config = initialize_hass_handler(conn)
    api_key = ha_config.get("api_key")
    base_url = ha_config.get("base_url")
    """
    state = { "type":"brightness_up","input":"80","is_muted":"true"}
    """
    domains = entity_id.split(".")
    if len(domains) > 1:
        domain = domains[0]
    else:
        return "执行失败，错误的设备id"
    action = ""
    arg = ""
    value = ""
    if state["type"] == "turn_on":
        description = "设备已打开"
        if domain == "cover":
            action = "open_cover"
        elif domain == "vacuum":
            action = "start"
        else:
            action = "turn_on"
    elif state["type"] == "turn_off":
        description = "设备已关闭"
        if domain == "cover":
            action = "close_cover"
        elif domain == "vacuum":
            action = "stop"
        else:
            action = "turn_off"
    elif state["type"] == "brightness_up":
        description = "灯光已调亮"
        action = "turn_on"
        arg = "brightness_step_pct"
        value = 10
    elif state["type"] == "brightness_down":
        description = "灯光已调暗"
        action = "turn_on"
        arg = "brightness_step_pct"
        value = -10
    elif state["type"] == "brightness_value":
        description = f"亮度已调整到{state['input']}"
        action = "turn_on"
        arg = "brightness_pct"
        value = state["input"]
    elif state["type"] == "set_color":
        description = f"颜色已调整到{state['rgb_color']}"
        action = "turn_on"
```

```
    arg = "rgb_color"
    value = state["rgb_color"]
elif state["type"] == "set_kelvin":
    description = f"色温已调整到{state['input']}K"
    action = "turn_on"
    arg = "kelvin"
    value = state["input"]
elif state["type"] == "volume_up":
    description = "音量已调大"
    action = state["type"]
elif state["type"] == "volume_down":
    description = "音量已调小"
    action = state["type"]
elif state["type"] == "volume_set":
    description = f"音量已调整到{state['input']}"
    action = state["type"]
    arg = "volume_level"
    value = state["input"]
    if state["input"] >= 1:
        value = state["input"] / 100
elif state["type"] == "volume_mute":
    description = f"设备已静音"
    action = state["type"]
    arg = "is_volume_muted"
    value = state["is_muted"]
elif state["type"] == "pause":
    description = f"设备已暂停"
    action = state["type"]
    if domain == "media_player":
        action = "media_pause"
    if domain == "cover":
        action = "stop_cover"
    if domain == "vacuum":
        action = "pause"
elif state["type"] == "continue":
    description = f"设备已继续"
    if domain == "media_player":
        action = "media_play"
    if domain == "vacuum":
        action = "start"
else:
    return f"{domain} {state['type']}功能尚未支持"

if arg == "":
    data = {
        "entity_id": entity_id,
    }
else:
    data = {"entity_id": entity_id, arg: value}
url = f"{base_url}/api/services/{domain}/{action}"
headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}

async with httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=3.0)) as client:
    response = await client.post(url, headers=headers, json=data)

logger.bind(tag=TAG).info(
    f"设置状态:{description},url:{url},return_code:{response.status_code}"
)
if response.status_code == 200:
    return description
```

```
    else:
        return f"设置失败，错误码: {response.status_code}"
# ===== 文件: xiaozhi-server/plugins_func/functions/play_music.py =====
import os
import re
import time
import random
import difflib
import traceback
from pathlib import Path
from core.providers.tts.dto.dto import TTSMessageDTO, SentenceType, ContentType
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__

MUSIC_CACHE = {}

play_music_function_desc = {
    "type": "function",
    "function": {
        "name": "play_music",
        "description": "当用户要求播放音乐、歌曲时调用。",
        "parameters": {
            "type": "object",
            "properties": {
                "song_name": {
                    "type": "string",
                    "description": "歌曲名称，如果用户没有指定具体歌名则为'random', 明确指定的时返回音乐
    的名字 示例: ```用户:播放两只老虎\n参数：两只老虎``` ```用户:播放音乐 \n参数：random ```",
                }
            },
            "required": ["song_name"],
        },
    },
}


@register_function("play_music", play_music_function_desc, ToolType.SYSTEM_CTL)
async def play_music(conn: "ConnectionHandler", song_name: str):
    try:
        music_intent = (
            f"播放音乐 {song_name}" if song_name != "random" else "随机播放音乐"
        )
        await handle_music_command(conn, music_intent)
        return ActionResponse(
            action=Action.RECORD, result="指令已接收", response="正在为您播放音乐"
        )
    except Exception as e:
        conn.logger.bind(tag=TAG).error(f"处理音乐意图错误: {e}")
        return ActionResponse(
            action=Action.RESPONSE, result=str(e), response="播放音乐时出错了"
        )


def _extract_song_name(text):
    """从用户输入中提取歌名"""
```

```
    for keyword in ["播放音乐"]:
        if keyword in text:
            parts = text.split(keyword)
            if len(parts) > 1:
                return parts[1].strip()
    return None


def _find_best_match(potential_song, music_files):
    """查找最匹配的歌曲"""
    best_match = None
    highest_ratio = 0

    for music_file in music_files:
        song_name = os.path.splitext(music_file)[0]
        ratio = difflib.SequenceMatcher(None, potential_song, song_name).ratio()
        if ratio > highest_ratio and ratio > 0.4:
            highest_ratio = ratio
            best_match = music_file
    return best_match


def get_music_files(music_dir, music_ext):
    music_dir = Path(music_dir)
    music_files = []
    music_file_names = []
    for file in music_dir.rglob("*"):
        # 判断是否是文件
        if file.is_file():
            # 获取文件扩展名
            ext = file.suffix.lower()
            # 判断扩展名是否在列表中
            if ext in music_ext:
                # 添加相对路径
                music_files.append(str(file.relative_to(music_dir)))
                music_file_names.append(
                    os.path.splitext(str(file.relative_to(music_dir)))[0]
                )
    return music_files, music_file_names


def initialize_music_handler(conn: "ConnectionHandler"):
    global MUSIC_CACHE
    if MUSIC_CACHE == {}:
        plugins_config = conn.config.get("plugins", {})
        if "play_music" in plugins_config:
            MUSIC_CACHE["music_config"] = plugins_config["play_music"]
            MUSIC_CACHE["music_dir"] = os.path.abspath(
                MUSIC_CACHE["music_config"].get("music_dir", "./music")  # 默认路径修改
            )
            MUSIC_CACHE["music_ext"] = MUSIC_CACHE["music_config"].get(
                "music_ext", (".mp3", ".wav", ".p3")
            )
            MUSIC_CACHE["refresh_time"] = MUSIC_CACHE["music_config"].get(
                "refresh_time", 60
            )
        else:
            MUSIC_CACHE["music_dir"] = os.path.abspath("./music")
            MUSIC_CACHE["music_ext"] = (".mp3", ".wav", ".p3")
            MUSIC_CACHE["refresh_time"] = 60
```

```
        # 获取音乐文件列表
        MUSIC_CACHE["music_files"], MUSIC_CACHE["music_file_names"] = get_music_files(
            MUSIC_CACHE["music_dir"], MUSIC_CACHE["music_ext"]
        )
        MUSIC_CACHE["scan_time"] = time.time()
    return MUSIC_CACHE


async def handle_music_command(conn: "ConnectionHandler", text):
    initialize_music_handler(conn)
    global MUSIC_CACHE

    """处理音乐播放指令"""
    clean_text = re.sub(r"[^\w\s]", "", text).strip()
    conn.logger.bind(tag=TAG).debug(f"检查是否是音乐命令: {clean_text}")

    # 尝试匹配具体歌名
    if os.path.exists(MUSIC_CACHE["music_dir"]):
        if time.time() - MUSIC_CACHE["scan_time"] > MUSIC_CACHE["refresh_time"]:
            # 刷新音乐文件列表
            MUSIC_CACHE["music_files"], MUSIC_CACHE["music_file_names"] = (
                get_music_files(MUSIC_CACHE["music_dir"], MUSIC_CACHE["music_ext"])
            )
            MUSIC_CACHE["scan_time"] = time.time()

        potential_song = _extract_song_name(clean_text)
        if potential_song:
            best_match = _find_best_match(potential_song, MUSIC_CACHE["music_files"])
            if best_match:
                conn.logger.bind(tag=TAG).info(f"找到最匹配的歌曲: {best_match}")
                await play_local_music(conn, specific_file=best_match)
                return True
    # 检查是否是通用播放音乐命令
    await play_local_music(conn)
    return True


def _get_random_play_prompt(song_name):
    """生成随机播放引导语"""
    # 移除文件扩展名
    clean_name = os.path.splitext(song_name)[0]
    prompts = [
        f"正在为您播放，《{clean_name}》",
        f"请欣赏歌曲，《{clean_name}》",
        f"即将为您播放，《{clean_name}》",
        f"现在为您带来，《{clean_name}》",
        f"让我们一起聆听，《{clean_name}》",
        f"接下来请欣赏，《{clean_name}》",
        f"此刻为您献上，《{clean_name}》",
    ]
    # 直接使用random.choice，不设置seed
    return random.choice(prompts)


async def play_local_music(conn: "ConnectionHandler", specific_file=None):
    global MUSIC_CACHE
    """播放本地音乐文件"""
    try:
        if not os.path.exists(MUSIC_CACHE["music_dir"]):
            conn.logger.bind(tag=TAG).error(
```

```
                f"音乐目录不存在: " + MUSIC_CACHE["music_dir"]
            )
            return

        # 确保路径正确性
        if specific_file:
            selected_music = specific_file
            music_path = os.path.join(MUSIC_CACHE["music_dir"], specific_file)
        else:
            if not MUSIC_CACHE["music_files"]:
                conn.logger.bind(tag=TAG).error("未找到MP3音乐文件")
                return
            selected_music = random.choice(MUSIC_CACHE["music_files"])
            music_path = os.path.join(MUSIC_CACHE["music_dir"], selected_music)

        if not os.path.exists(music_path):
            conn.logger.bind(tag=TAG).error(f"选定的音乐文件不存在: {music_path}")
            return
        text = _get_random_play_prompt(selected_music)
        conn.tts.store_tts_text(conn.sentence_id, text)
        # conn.dialogue.put(Message(role="assistant", content=text))

        if conn.intent_type == "intent_llm":
            conn.tts.tts_text_queue.put(
                TTSMessageDTO(
                    sentence_id=conn.sentence_id,
                    sentence_type=SentenceType.FIRST,
                    content_type=ContentType.ACTION,
                )
            )
        conn.tts.tts_text_queue.put(
            TTSMessageDTO(
                sentence_id=conn.sentence_id,
                sentence_type=SentenceType.MIDDLE,
                content_type=ContentType.TEXT,
                content_detail=text,
            )
        )
        conn.tts.tts_text_queue.put(
            TTSMessageDTO(
                sentence_id=conn.sentence_id,
                sentence_type=SentenceType.MIDDLE,
                content_type=ContentType.FILE,
                content_file=music_path,
            )
        )
        if conn.intent_type == "intent_llm":
            conn.tts.tts_text_queue.put(
                TTSMessageDTO(
                    sentence_id=conn.sentence_id,
                    sentence_type=SentenceType.LAST,
                    content_type=ContentType.ACTION,
                )
            )

    except Exception as e:
        conn.logger.bind(tag=TAG).error(f"播放音乐失败: {str(e)}")
        conn.logger.bind(tag=TAG).error(f"详细错误: {traceback.format_exc()}")
# ===== 文件: xiaozhi-server/plugins_func/functions/search_from_ragflow.py =====
import json
```

```
import httpx
from config.logger import setup_logging
from plugins_func.register import register_function, ToolType, ActionResponse, Action
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

# 定义基础的函数描述模板
SEARCH_FROM_RAGFLOW_FUNCTION_DESC = {
    "type": "function",
    "function": {
        "name": "search_from_ragflow",
        "description": "从知识库中查询信息",
        "parameters": {
            "type": "object",
            "properties": {"question": {"type": "string", "description": "查询的问题"}},
            "required": ["question"],
        },
    },
}


@register_function(
    "search_from_ragflow", SEARCH_FROM_RAGFLOW_FUNCTION_DESC, ToolType.SYSTEM_CTL
)
async def search_from_ragflow(conn: "ConnectionHandler", question=None):
    # 确保字符串参数正确处理编码
    if question and isinstance(question, str):
        # 确保问题参数是UTF-8编码的字符串
        pass
    else:
        question = str(question) if question is not None else ""

    ragflow_config = conn.config.get("plugins", {}).get("search_from_ragflow", {})
    base_url = ragflow_config.get("base_url", "")
    api_key = ragflow_config.get("api_key", "")
    dataset_ids = ragflow_config.get("dataset_ids", [])

    url = base_url + "/api/v1/retrieval"
    headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}

    # 确保payload中的字符串都是UTF-8编码
    payload = {"question": question, "dataset_ids": dataset_ids}

    try:
        # 使用ensure_ascii=False确保JSON序列化时正确处理中文
        async with httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=3.0), verify=False) as client:
            response = await client.post(url, json=payload, headers=headers)

        # 显式设置响应的编码为utf-8
        response.encoding = "utf-8"

        response.raise_for_status()

        # 先获取文本内容，然后手动处理JSON解码
        response_text = response.text
```

```
    result = json.loads(response_text)

    if result.get("code") != 0:
        error_detail = result.get("error", {}).get("detail", "未知错误")
        error_message = result.get("error", {}).get("message", "")
        error_code = result.get("code", "")

        # 安全地记录错误信息
        logger.bind(tag=TAG).error(
            f"RAGFlow API调用失败，响应码：{error_code}，错误详情：{error_detail}，完整响应：{result
}"
        )

        # 构建详细的错误响应
        error_response = f"RAG接口返回异常（错误码：{error_code}）"

        if error_message:
            error_response += f"：{error_message}"
        if error_detail:
            error_response += f"\n详情：{error_detail}"

        return ActionResponse(Action.RESPONSE, None, error_response)

    chunks = result.get("data", {}).get("chunks", [])
    contents = []
    for chunk in chunks:
        content = chunk.get("content", "")
        if content:
            # 安全地处理内容字符串
            if isinstance(content, str):
                contents.append(content)
            elif isinstance(content, bytes):
                contents.append(content.decode("utf-8", errors="replace"))
            else:
                contents.append(str(content))

    if contents:
        # 组织知识库内容为引用模式
        context_text = f"# 关于问题【{question}】查到知识库如下\n"
        context_text += "```\n\n\n".join(contents[:5])
        context_text += "\n```"
    else:
        context_text = "根据知识库查询结果，没有相关信息。"
    return ActionResponse(Action.REQLLM, context_text, None)

except httpx.TimeoutException as e:
    error_response = "RAG接口请求超时"
    error_response += "\n可能原因：RAGflow服务响应缓慢或网络延迟"
    error_response += "\n解决方案：请稍后重试或检查RAGflow服务性能"
    return ActionResponse(Action.RESPONSE, None, error_response)

except httpx.HTTPStatusError as e:
    if hasattr(e.response, "status_code"):
        status_code = e.response.status_code
        error_response = f"RAG接口HTTP错误（状态码：{status_code}）"
        try:
            error_detail = e.response.json().get("error", {}).get("message", "")
            if error_detail:
                error_response += f"\n错误详情：{error_detail}"
```

```
            except:
                pass
        else:
            error_response = f"RAG接口HTTP异常：{str(e)}"
        return ActionResponse(Action.RESPONSE, None, error_response)

    except httpx.HTTPError as e:
        error_response = "无法连接到RAG接口"
        error_response += "\n可能原因：RAGflow服务地址错误或服务未运行"
        error_response += "\n解决方案：请检查RAGflow服务地址配置和服务状态"
        return ActionResponse(Action.RESPONSE, None, error_response)

    except Exception as e:
        # 其他异常
        error_type = type(e).__name__
        logger.bind(tag=TAG).error(
            f"RAGflow处理异常，异常类型：{error_type}，详情：{str(e)}"
        )

        # 提供详细的错误信息
        error_response = f"RAG接口处理异常（{error_type}）：{str(e)}"
        return ActionResponse(Action.RESPONSE, None, error_response)
# ===== 文件: xiaozhi-server/plugins_func/functions/web_search.py =====
import httpx
from config.logger import setup_logging
from plugins_func.register import (
    register_function,
    ToolType,
    ActionResponse,
    Action,
)
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from core.connection import ConnectionHandler

TAG = __name__
logger = setup_logging()

_DEFAULT_DESCRIPTION = (
    "联网搜索工具。当用户明确需要联网搜索问题时使用此工具。"
)

WEB_SEARCH_FUNCTION_DESC = {
    "type": "function",
    "function": {
        "name": "web_search",
        "description": _DEFAULT_DESCRIPTION,
        "parameters": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "搜索关键词或问题",
                }
            },
            "required": ["query"],
        },
    },
}
```

```
async def _search_metaso(api_key: str, query: str, max_results: int) -> str:
    """调用秘塔搜索API"""
    url = "https://metaso.cn/api/v1/search"
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
    }
    payload = {
        "q": query,
        "size": max_results,
        "stream": False,
        "scope": "webpage",
        "includeSummary": True,
        "includeRawContent": False,
        "conciseSnippet": False,
    }
    logger.bind(tag=TAG).debug(f"秘塔搜索请求 | URL: {url} | payload: {payload}")
    async with httpx.AsyncClient(timeout=httpx.Timeout(15.0, connect=3.0)) as client:
        response = await client.post(url, json=payload, headers=headers)
    data = response.json()
    logger.bind(tag=TAG).debug(f"秘塔搜索响应 | status: {response.status_code}")

    webpages = data.get("webpages", [])
    if not webpages:
        return "未找到相关搜索结果。"

    lines = ["【联网搜索结果】"]
    for i, item in enumerate(webpages, 1):
        title = item.get("title", "无标题")
        snippet = item.get("summary", "")
        date = item.get("date", "")
        lines.append(f"{i}. 标题：{title}")
        if date:
            lines.append(f"   日期：{date}")
        if snippet:
            lines.append(f"   摘要：{snippet}")

    return "\n".join(lines)


async def _search_tavily(api_key: str, query: str, max_results: int) -> str:
    """调用Tavily搜索API"""
    url = "https://api.tavily.com/search"
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
    }
    payload = {
        "query": query,
        "max_results": max_results,
        "search_depth": "advanced",
        "include_answer": "advanced",
    }
    logger.bind(tag=TAG).debug(f"Tavily搜索请求 | URL: {url} | payload: {payload}")
    async with httpx.AsyncClient(timeout=httpx.Timeout(15.0, connect=3.0)) as client:
        response = await client.post(url, json=payload, headers=headers)
    data = response.json()
    logger.bind(tag=TAG).debug(f"Tavily搜索响应 | status: {response.status_code} | data: {data}")
```

```
    results = data.get("results", [])
    if not results:
        return "未找到相关搜索结果。"

    answer = data.get("answer", "")
    lines = [f"【联网搜索结果】\n总结：{answer}"]
    # for i, item in enumerate(results, 1):
    #     title = item.get("title", "无标题")
    #     summary = item.get("content", "")
    #     lines.append(f"{i}. 标题：{title}")
    #     if summary:
    #         lines.append(f"   摘要：{summary}")

    return "\n".join(lines)


@register_function("web_search", WEB_SEARCH_FUNCTION_DESC, ToolType.SYSTEM_CTL)
async def web_search(conn: "ConnectionHandler", query: str = None):
    logger.bind(tag=TAG).info(f"web_search 被调用 | query={query}")
    if not query:
        return ActionResponse(Action.REQLLM, "请提供搜索关键词。", None)

    web_search_config = conn.config.get("plugins", {}).get("web_search", {})
    provider = web_search_config.get("provider", "").lower()
    max_results = int(web_search_config.get("max_results", 3))
    logger.bind(tag=TAG).info(f"web_search 配置 | provider={provider} | max_results={max_results} | conf
    ig_keys={list(web_search_config.keys())}")

    api_key = web_search_config.get("api_key", "")
    if not api_key:
        return ActionResponse(
            Action.REQLLM,
            "联网搜索功能未配置API Key，请在配置文件中填写。",
            None,
        )

    try:
        if provider == "metaso":
            result_text = await _search_metaso(api_key, query, max_results)
        elif provider == "tavily":
            result_text = await _search_tavily(api_key, query, max_results)
        else:
            return ActionResponse(
                Action.REQLLM,
                f"联网搜索功能未配置或配置的搜索源无效（当前：{provider}），请检查配置。",
                None,
            )
        logger.bind(tag=TAG).info(f"搜索结果组装完成:\n{result_text}")
    except httpx.TimeoutException:
        logger.bind(tag=TAG).error("联网搜索请求超时")
        result_text = "联网搜索请求超时，请稍后重试。"
    except httpx.HTTPStatusError as e:
        logger.bind(tag=TAG).error(f"联网搜索请求失败: {e}")
        result_text = "联网搜索请求失败，请稍后重试。"
    except Exception as e:
        logger.bind(tag=TAG).error(f"联网搜索异常: {e}")
        result_text = "联网搜索出现异常，请稍后重试。"

    return ActionResponse(Action.REQLLM, result_text, None)
```